

Graph Neural Networks for the Offline Nanosatellite Task Scheduling Problem

Bruno Machado Pacheco^a, Laio Oriel Seman^a, Cezar Antônio Rigo^b, Eduardo Camponogara^a, Eduardo Augusto Bezerra^b, Leandro dos Santos Coelho^c

^a*Department of Automation and Systems Engineering, Federal University of Santa Catarina (UFSC), Florianópolis, SC, Brazil*

^b*Department of Electrical Engineering, Federal University of Santa Catarina (UFSC), Florianópolis, SC, Brazil*

^c*Department of Electrical Engineering, Federal University of Parana (UFPR), Curitiba, PR, Brazil*

Abstract

This study investigates how to schedule nanosatellite tasks more efficiently using Graph Neural Networks (GNNs). In the Offline Nanosatellite Task Scheduling (ONTS) problem, the goal is to find the optimal schedule for tasks to be carried out in orbit while taking into account Quality-of-Service (QoS) considerations such as priority, minimum and maximum activation events, execution time-frames, periods, and execution windows, as well as constraints on the satellite's power resources and the complexity of energy harvesting and management. This study explores the use of Graph Neural Networks (GNNs) as primal heuristics for the ONTS problem, as this class of deep learning model has been effectively applied to optimization problems such as the traveling salesman, scheduling, and facility placement. We investigate whether GNNs can learn the complex structure of the ONTS problem with respect to feasibility and optimality of candidate solutions. Furthermore, we evaluate using GNN-based heuristic solutions to provide better solutions (w.r.t. the objective value) to the ONTS problem and reduce the optimization cost. Our experiments show that GNNs are not only able to learn feasibility and optimality for instances of the ONTS problem, but they can generalize to harder instances than those seen during training, addressing the data acquisition cost of training deep learning models on optimization problems. On top of that, the GNN-based heuristics improved the expected objective value of the best solution found under the time limit in 45%, and reduced the expected time to find a feasible solution in 35%, when compared to the SCIP (Solving Constraint Integer Programs) solver in its off-the-shelf configuration.

Keywords: Scheduling, Graph Neural Network, Combinatorial Optimization, Nanosatellite, Deep Learning

Email addresses: bruno.m.pacheco@posgrad.ufsc.br (Bruno Machado Pacheco), laio@ieee.org (Laio Oriel Seman), cez.ar.a.rigo@gmail.com (Cezar Antônio Rigo), eduardo.camponogara@ufsc.br (Eduardo Camponogara), eduardo.bezerra@ufsc.br (Eduardo Augusto Bezerra), leandro.coelho@ufpr.br (Leandro dos Santos Coelho)

1. Introduction

Nanosatellites have attracted attention from industry and academia for over a decade, primarily due to their reduced development and launch costs (Shiroma et al., 2011; Lucia et al., 2021; Nagel et al., 2020; Saeed et al., 2020). However, these miniaturized spacecraft face significant operational challenges due to their inherent resource constraints, particularly in task scheduling and resource management.

The Offline Nanosatellite Task Scheduling (ONTS) problem addresses these challenges by optimizing the allocation of satellite resources during the mission planning phase. This optimization is particularly complex due to the diverse operational characteristics of satellite payloads, including varying power consumption profiles, data handling requirements, and specific operational windows. Furthermore, the reliance on solar power as the primary energy source introduces considerable variability in energy supply, affected by factors such as cyclical eclipses in Low Earth Orbit (LEO), varying solar incidence angles, spacecraft attitude, and size.

As an offline planning tool, ONTS assists engineers in simulating and evaluating performance before launch. The problem must address five fundamental scheduling requirements: *i*) non-preemptive tasks that must be completed without interruption once initiated; *ii*) task periodicity within specific thresholds; *iii*) minimum and maximum task startups within an orbit to meet Quality-of-Service (QoS) requirements; *iv*) task execution windows for precise operations; and *v*) battery state-of-charge management balancing energy consumption with generation.

Traditional approaches to solving the ONTS problem have employed various mathematical optimization techniques, progressing from Integer Programming (IP) (Rigo et al., 2021b) to Mixed-Integer Linear Programming (MILP) (Rigo et al., 2021a; Seman et al., 2022) and Continuous-Time formulations (Camponogara et al., 2022). However, due to the (Non-deterministic Polynomial time)-hard nature of the problem, efficiently solving large instances—involving many tasks or extended planning horizons—remains an open research challenge.

Meanwhile, several recent investigations (Bengio et al., 2021; Karimi-Mamaghan et al., 2022; Pacheco et al., 2023a; Han et al., 2023) have considered machine learning tools to address combinatorial optimization problems, such as the single machine problem (Parmentier and T’Kindt, 2023), resource-constrained project scheduling (Guo et al., 2023), and knapsack problems (Yang et al., 2022). Graph Neural Networks (GNNs), in particular, have gained popularity in recent years to solve combinatorial optimization problems when the underlying structure can be represented as a graph (Zhang et al., 2023), which is the case for MILP (Khalil et al., 2022).

This paper investigates the novel application of GNNs for the ONTS problem. More specifically, we consider two research questions:

- Can a graph neural network learn the structure of the ONTS problem?

- Is a GNN-based heuristic effective (fast and accurate) for the ONTS problem?

To address these two questions, we propose two sets of experiments. First, we train GNNs to predict the feasibility and optimality of candidate solutions given an instance of the problem. In the second set of experiments, we train GNNs to generate candidate solutions given problem instances and use the resulting models to build matheuristics (Boschetti and Maniezzo, 2022). Our results indicate that the proposed GNN can learn the feasibility and optimality of candidate solutions, even when given instances that are larger (and harder) than those seen during training. Furthermore, GNN-based matheuristics effectively provide high-quality solutions for large problems, overcoming the MILP solver (SCIP) both in the time to generate feasible solutions and the quality of the solutions found within limited time.

Everything considered, this paper takes a fundamentally different approach compared to previous works. Rather than proposing a new standalone algorithm, our work aims to develop a supportive heuristic component that can enhance existing solution methods. This is accomplished through three main aspects: *(i)* the GNN-based heuristic is designed specifically to improve and accelerate the performance of traditional exact methods or sophisticated ones like branch-and-price, not to replace them; *(ii)* it serves as an integrated component that works in conjunction with established algorithms, providing high-quality initial solutions to speed up convergence; and *(iii)* it maintains the mathematical guarantees of the host algorithm while improving its computational efficiency. In summary, the main contributions of this paper are as follows:

- The introduction of SatGNN, a pioneering GNN architecture suitable to the ONTS problem;
- A showcase of SatGNN’s impressive performance in accurately classifying feasibility, predicting optimality, and generating effective heuristic solutions;
- A demonstration of GNN’s robust generalization abilities for optimization problems, particularly when dealing with larger ONTS instances;
- An implication of the potential for a transformative impact on complex space mission scheduling.

The remainder of this paper is organized as follows: Section 2 provides an overview of the related literature. Section 3 describes the problem in detail, providing context and background information and formulating the optimization problem. Section 4 provides the necessary theoretical background on GNNs and their application to linear optimization problems. The data acquisition, the proposed GNN architecture, and the heuristics for optimization problems are presented in Section 5. The experiments that tackle the two research questions and the results are presented in Section 6. Finally, Sections 7 and 8 discuss the key findings and provide concluding remarks and future research directions.

2. Related Work

The ONTS problem has received significant contributions in recent years, with different approaches and formulations. At the same time, machine learning applications for optimization problems have become a prolific area of research. This section briefly overviews our paper’s most influential works, which are also summarized in Table 1.

Nanosatellite task scheduling

Mathematical programming is consolidated as a practical approach for satellite scheduling problems. Wang et al. (2013) formulate the problem of dynamic scheduling emergency tasks as a mathematical program with multiple objectives. Wang et al. (2016) handle the uncertainties from cloud blockage with respect to the satellite visibility and propose an approximate integer linear program to schedule tasks, which is solved by the authors using a branch and cut approach. Considering multiple satellites, Niu et al. (2018) propose an integer program model and a genetic algorithm to provide feasible solutions faster under emergency conditions.

Targeting small satellites, Rouzot et al. (2024) formulated the problem using constraint programming to generate schedules that satisfy the multiple tight requirements that a nanosatellite mission imposes. Rigo et al. (2021b) formulated the problem of maximizing task allocation under power availability constraints as an integer programming (IP) problem, enabling the generation of optimal schedules. Seman et al. (2022) extended the approach by Rigo et al. (2021b) and proposed a Mixed-Integer Linear Programming (MILP) formulation to tackle the task scheduling in constellations of nanosatellites. Camponogara et al. (2022) applied the Multi-Operation Sequencing with Synchronized Start Times (MOS-SST) representation to the problem, maintaining an MILP formulation but extending the problem to continuous time. The evolution of formulations and methodologies highlights the continuous efforts to find the most efficient and effective solutions to the ONTS problem.

More recently, given the difficulty in solving complex instances of the ONTS problem, Rigo et al. (2022) proposed a Dantzig-Wolfe decomposition, coupled with a branch-and-price (B&P) methodology, which was further adapted in a heuristic solution through the use of a genetic algorithm (Seman et al., 2023). This approach was designed to build a unique column-based formulation that produces feasible and optimal schedules. In addition, they leveraged the Dynamic Programming (DP) technique to identify optimal columns and speed up the whole process. Their computational experiments significantly improved overall solution time compared to a commercial MILP solver, with a 70% reduction in computation time. Despite these promising results, the B&P method introduces specific challenges and limitations that preclude its universal application. One significant drawback is its heavy reliance on the suitability of the decomposition, which is notoriously problem-specific. Currently, no known method allows practitioners to *a priori* ascertain the presence of a decomposable structure in a given problem (Kruber et al., 2017). This lack of a deterministic approach to identifying suitable problems adds a layer of complexity to the application of B&P. Additionally, the complexity inherent in the decomposition and the optimization algorithm can result in a high development undertaking. The intricate nature of these processes requires

Table 1: Summary of related work

Reference	Main Contribution	Methodology	Application
<i>Nanosatellite Task Scheduling</i>			
Wang et al. (2013)	Dynamic scheduling of emergency tasks	Mathematical programming with multiple objectives	Emergency tasks
Wang et al. (2016)	Handling cloud blockage uncertainties	Approximate integer linear programming	Visibility constraints
Niu et al. (2018)	Multi-satellite scheduling under emergency conditions	Integer programming + genetic algorithm	Emergency response
Rouzot et al. (2024)	Scheduling under tight nanosatellite constraints	Constraint programming	Nanosatellite missions
Rigo et al. (2021b)	Power-aware task allocation	Integer programming	Power optimization
Seman et al. (2022)	Constellation task scheduling	Mixed-integer linear programming	Nanosatellite constellations
Rigo et al. (2022)	Column generation approach	Branch-and-price	Task scheduling
<i>Machine Learning for Scheduling</i>			
Hopfield and Tank (1985)	First ML application to optimization	Hopfield neural networks	TSP
Wang et al. (2021)	Heuristic scheduling solutions	Reinforcement learning (MDP)	General scheduling
Feng et al. (2024)	Static constraint satisfaction	Deep learning + optimization	Satellite scheduling
<i>GNNs for Combinatorial Optimization</i>			
Khalil et al. (2017)	Graph-based CO solutions	GNN-based greedy heuristic	Graph problems
Gasse et al. (2019)	MILP problem embedding	Bipartite graph representation	General MILP
Ding et al. (2020)	Variable selection in B&B	GNN-guided branching	MILP solving
Nair et al. (2020)	Neural Diving approach	GNN-based partial solutions	MILP solving
Han et al. (2023)	Trust region optimization	GNN-guided solution space exploration	MILP solving

specialized knowledge and expertise, potentially rendering the B&P method prohibitive for particular applications or settings. Consequently, while the B&P methodology offers substantial improvements in computational efficiency for specific problem instances, its applicability may be limited by these challenges, emphasizing the need for caution in considering it as a universal solution for all optimization problems.

The literature demonstrates a clear progression in solution methodologies for the ONTS problem, from classical mathematical programming to sophisticated decomposition approaches. While the B&P methodology has demonstrated substantial computational advantages, its effectiveness remains contingent upon problem-specific characteristics and implementation expertise. These limitations underscore the necessity for developing more generalizable solution approaches that maintain computational efficiency while reducing implementation complexity.

Machine learning for offline task scheduling

Machine learning applications for optimization problems were first proposed in 1985, applying Hopfield neural networks to the Travelling Salesperson Problem (TSP) (Hopfield and Tank, 1985). Due to the hardware limitations of the time (Smith, 1999), neural network applications to optimization problems have stayed out of the spotlight up to recent years, when developments in novel architectures, more powerful hardware, and promising published results have attracted the attention of the research community back to this area (Bengio et al., 2021).

For scheduling problems, many authors have proposed reinforcement learning strategies to make the scheduling decisions, even in offline settings. A series of works have focused on training models to provide completely heuristic solutions, modeling the scheduling problem as a markov decision process (Wang et al., 2021; Tassel et al., 2021; Zhang et al., 2020). Another research direction with significant contributions has been on modeling uncertainty through reinforcement learning and modifying the parameters of the scheduling IP formulation based on the model’s output (Kenworthy et al., 2022; de Magalhães Taveira-Gomes, 2017). To the best of our knowledge, no works in scheduling have proposed learning-based matheuristics, for example, as is done for vertex coloring and set covering in Kruber et al. (2017).

This is particularly true for satellite task scheduling problems, where many works focused on reinforcement learning policies for generating a heuristic solution for the schedule (Li et al., 2023; Huang et al., 2021; Cui et al., 2023; Dalin et al., 2021). To the best of our knowledge, the work by Feng et al. (2024) is the only one to use a deep learning model interacting directly with the solver of a satellite task scheduling problem. The authors train a deep learning model to solve the problem’s static constraints, alleviating the optimizer’s job, which aims to satisfy the dynamic constraints of the problem.

The applications of machine learning in scheduling problems have predominantly focused on reinforcement learning strategies, both for direct heuristic solutions and uncertainty modeling. However, the integration of learning-based matheuristics with satellite task scheduling remains largely unexplored, particularly in contrast to other combinatorial optimization domains. This gap in the literature presents opportunities for novel methodological contributions.

One of the drivers of the recent interest in machine learning for Combinatorial Optimization (CO) problems was the proposition of structured artificial neural networks able to handle the symmetries that optimization problems have (Cappart et al., 2022), such as the invariance to permutation of constraints. A significant example of such property for CO is the work of Khalil et al. (2017), which developed a greedy heuristic for CO problems on graphs using a GNN. Gasse et al. (2019) proposed to embed linear programming problems as a bipartite graph, generalizing the use of GNNs to any MILP. The results presented in these works indicate that GNNs can learn the structures of MILP problems.

Many authors proposed using GNNs to predict solution values, aiming at using the learned model in heuristic solutions. Ding et al. (2020) proposed using such GNNs to guide the Branch and Bound (B&B) algorithm towards the variables the model had the most difficulty learning. Khalil et al. (2022) use the GNNs trained to predict biases of the binary variables to perform node selection and warm starting. Nair et al. (2020), in their approach named *Neural Diving*, proposed to train the model to provide partial solutions, which generate sub-problems through variable fixing. The multiple sub-problems are then solved using off-the-shelf solvers. Han et al. (2023) compare the fixing approach of Nair et al. (2020) to their proposed approach of defining a trust region around the partial solution generated by the GNN, showing significant improvements over off-the-shelf solvers like SCIP and Gurobi. These works highlight how promising building GNN-based heuristic approaches to MILP can be.

The literature evidences the efficacy of GNN architectures in addressing combinatorial optimization problems, particularly through their integration with traditional exact methods such as B&B. Recent developments in solution prediction and variable fixing strategies have yielded promising computational results. Nevertheless, the application of these techniques remains an emerging field, with significant potential for theoretical and practical advances.

To the best of our knowledge, while metaheuristic approaches like genetic algorithms have been applied to nanosatellite task scheduling (Seman et al., 2023), our work is the first to propose a matheuristic specifically designed to enhance exact solution methods. Unlike metaheuristics that operate independently but cannot guarantee global optimality, our approach is engineered to provide high-quality initial solutions that accelerate the convergence of exact methods while maintaining their optimality guarantees. Furthermore, machine learning applications to MILP has only recently attracted intense attention, as confirmed by the novelty of the works presented above and the lack of learning-based components in open-source solvers such as SCIP and CPLEX. In other words, although promising, applying GNNs to MILP still needs to be well-understood and is not guaranteed to yield positive results. In this context, the present work also contributes to the area of machine learning and combinatorial optimization research by validating novel techniques that complement rather than replace exact methods.

3. Offline Nanosatellite Task Scheduling (ONTS)

Nanosatellite scheduling problems concern the decisions on each task’s start and finish time. The tasks usually require periodic execution and during restricted moments along the orbit. Besides time, energy availability through orbit is a crucial resource to consider. Figure 1 shows an example of optimal scheduling, in which each job is represented by a different color and the activation and deactivations are shown through the steps of the signals. Proper scheduling must account for energy management, so the tasks do not draw more energy than the system can provide and the battery is not depleted before the end of the mission. Energy management is a difficult task since the nanosatellite draws power from its solar panels, with the energy availability depending on the attitude of the nanosatellite (which affects the orientation of the solar panels) and the trajectory with respect to Earth’s shadow, as illustrated in Figure 2.

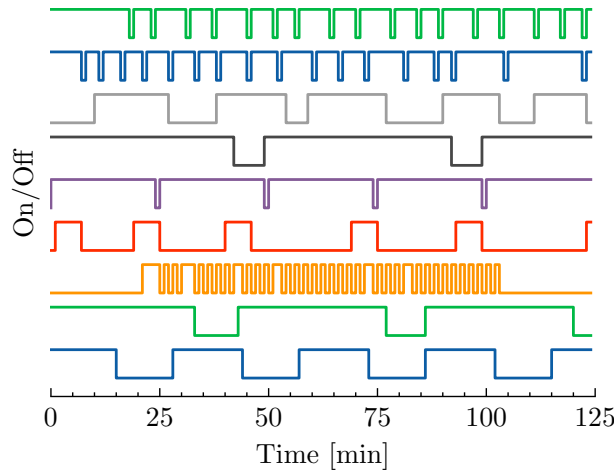


Figure 1: Illustration of an optimum scheduling for 9 tasks on a horizon of 125 time steps. Each color represents the execution of the different tasks.

3.1. MILP Formulation

A formulation that considers the realistic constraints and objectives of ONTS was proposed by Rigo et al. (2021b) and is presented here. For reference, all symbols (sets, variables, and parameters) used in our formulation are summarized in Appendix A. Given a set of jobs $\mathcal{J} = \{0, \dots, J\}$ that represents the tasks, and a set of time units $\mathcal{T} = \{0, \dots, T\}$ that represents the scheduling horizon, variables

$$\mathbf{x} = (x_{j,t})_{\substack{j=1,\dots,J \\ t=1,\dots,T}}$$

represent the allocation of jobs $j \in \mathcal{J}$ at times $t \in \mathcal{T}$, i.e., $x_{j,t} = 1$ indicates that job j is scheduled to execute at time t . Naturally, $x_{j,t}$ are binary variables, i.e., $\mathbf{x} \in \{0, 1\}^{JT}$.

It is assumed that there exists a priority for every job, which is defined by $\mathbf{u} = (u_1, \dots, u_J)$. The goal is to maximize the mission’s Quality of Service (QoS) (1), which is the sum of the

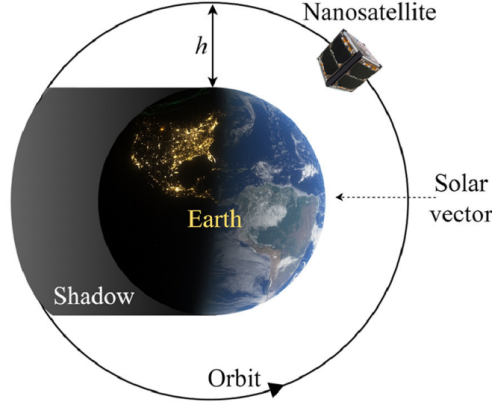


Figure 2: Illustration of a nanosatellite's orbit around Earth. Image from Rigo et al. (2022).

allocations weighted by the priorities,

$$\text{QoS}(\mathbf{x}; \mathbf{u}) = \sum_{j=1}^J \sum_{t=1}^T u_j x_{j,t} \quad (1)$$

To formalize the ONTS problem of maximizing QoS, we follow the formulation proposed by Rigo et al. (2021b). The following constraints are added to ensure that the scheduling

respects the requirements and specificities of each job:

$$\phi_{j,t} \geq x_{j,t}, \quad \forall j \in \mathcal{J}, t = 1 \quad (2a)$$

$$\phi_{j,t} \geq x_{j,t} - x_{j,(t-1)}, \quad \forall j \in \mathcal{J}, \forall t \in \mathcal{T} : t > 1 \quad (2b)$$

$$\phi_{j,t} \leq x_{j,t}, \quad \forall j \in \mathcal{J}, \forall t \in \mathcal{T} \quad (2c)$$

$$\phi_{j,t} \leq 2 - x_{j,t} - x_{j,(t-1)}, \quad \forall j \in \mathcal{J}, \forall t \in \mathcal{T} : t > 1 \quad (2d)$$

$$\sum_{t=1}^{w_j^{\min}} x_{j,t} = 0, \quad \forall j \in \mathcal{J} \quad (2e)$$

$$\sum_{t=w_j^{\max}+1}^T x_{j,t} = 0, \quad \forall j \in \mathcal{J} \quad (2f)$$

$$\sum_{l=t}^{t+t_j^{\min}-1} x_{j,l} \geq t_j^{\min} \phi_{j,t}, \quad \forall t \in \{1, \dots, T - t_j^{\min} + 1\}, \forall j \in \mathcal{J} \quad (2g)$$

$$\sum_{l=t}^{t+t_j^{\max}} x_{j,l} \leq t_j^{\max}, \quad \forall t \in \{1, \dots, T - t_j^{\max}\}, \forall j \in \mathcal{J} \quad (2h)$$

$$\sum_{l=t}^T x_{j,l} \geq (T - t + 1) \phi_{j,t}, \quad \forall t \in \{T - t_j^{\min} + 2, \dots, T\}, \forall j \in \mathcal{J} \quad (2i)$$

$$\sum_{l=t}^{t+p_j^{\min}-1} \phi_{j,l} \leq 1, \quad \forall t \in \{1, \dots, T - p_j^{\min} + 1\}, \forall j \in \mathcal{J} \quad (2j)$$

$$\sum_{l=t}^{t+p_j^{\max}-1} \phi_{j,l} \geq 1, \quad \forall t \in \{1, \dots, T - p_j^{\max} + 1\}, \forall j \in \mathcal{J} \quad (2k)$$

$$\sum_{t=1}^T \phi_{j,t} \geq y_j^{\min}, \quad \forall j \in \mathcal{J} \quad (2l)$$

$$\sum_{t=1}^T \phi_{j,t} \leq y_j^{\max}, \quad \forall j \in \mathcal{J} \quad (2m)$$

$$\phi_{j,t} \in \{0, 1\}, \quad \forall j \in \mathcal{J}, t \in \mathcal{T} \quad (2n)$$

$$x_{j,t} \in \{0, 1\}, \quad \forall j \in \mathcal{J}, t \in \mathcal{T}. \quad (2o)$$

Note that auxiliary binary variables

$$\phi = (\phi_{j,t})_{\substack{j=1,\dots,J \\ t=1,\dots,T}}$$

are used, which take a positive value if, and only if, job j was not running at time $t - 1$, but started running at time t . Constraints (2a) to (2d) are used to ensure this behavior. We

also consider that jobs may run only during a time window defined by parameters $w_j^{\min}$ and $w_j^{\max}$, which is ensured by constraints (2e) and (2f). Such behavior is necessary to ensure that a payload, for instance, runs only when passing above a certain territory.

Jobs may also have limits on continuous execution. If a job j starts running at time t , then it must run for at least $t_j^{\min}$ time steps, and at most $t_j^{\max}$ time steps. This is ensured by constraints (2g) and (2h). The formulation, through constraint (2i), also allows a job to start at the last time steps and keep running until the end, assuming it will keep running at the start of the following schedule.

A job may require to be executed periodically, at least every $p_j^{\min}$ time step, and at most every $p_j^{\max}$ time step. This is ensured through constraints (2j) and (2k), over the $\phi_{j,t}$ variables. A job may also require multiple executions through the planning horizon, starting at least $y_j^{\min}$ times, and at most $y_j^{\max}$ times, which is ensured through constraints (2l) and (2m).

Beyond job-specific restrictions in constraints (2), the formulation also covers energy management,

$$\sum_{j=1}^J q_j x_{j,t} \leq r_t + \gamma V_b, \quad \forall t \in \mathcal{T} \quad (3a)$$

$$b_t = r_t - \sum_{j \in \mathcal{J}} q_j x_{j,t}, \quad \forall t \in \mathcal{T} \quad (3b)$$

$$i_t = \frac{b_t}{V_b}, \quad \forall t \in \mathcal{T} \quad (3c)$$

$$\text{SoC}_{t+1} = \text{SoC}_t + \frac{i_t e}{60 Q}, \quad \forall t \in \mathcal{T} \quad (3d)$$

$$\text{SoC}_t \leq 1, \quad \forall t \in \mathcal{T} \quad (3e)$$

$$\text{SoC}_t \geq \rho, \quad \forall t \in \mathcal{T}. \quad (3f)$$

Parameter r_t provides the power available from the solar panels at time t and q_j the power required for executing job j . Thus, constraint (3a) limits the power consumption, with $\gamma \cdot V_b$ being the maximum power the battery can provide. Constraints (3b) to (3d) update the auxiliary variables b_t and SoC_t , which represent the exceeding power and State of Charge (SoC) at time t , based on the battery capacity Q and the discharge efficiency e . The SoC of the battery must stay within the limits given in constraints (3e) and (3f), with ρ being a lower limit usually greater than zero as a safety measure.

Thus, the MILP formulation is the maximization of (1) while subject to constraints (2) and (3). In other words, we can write any instance $I \in \mathcal{I}$, where $\mathcal{I}$ is the set of all possible

instances of the ONTS problem, as

$$\begin{aligned}
I : \max_{\mathbf{x}, \boldsymbol{\phi}, \text{SoC}} & \underbrace{\sum_{j=1}^J \sum_{t=1}^T u_j x_{j,t}}_{\text{QoS}} \\
\text{s.t. } & (2), (3) \\
& \mathbf{x}, \boldsymbol{\phi} \in \{0, 1\}^{JT}, \text{SoC} \in [0, 1]^T.
\end{aligned} \tag{4}$$

Note that the continuous variables SoC can be completely determined by the binary variables $\mathbf{x}$ and $\boldsymbol{\phi}$, so our problem can be reduced to finding an assignment $\mathbf{z} = (\mathbf{x}, \boldsymbol{\phi}) \in \{0, 1\}^{2JT}$.

Any instance $I \in \mathcal{I}$ is parameterized by the number of tasks J , the number of time units T and the parameters $\mathbf{u}, \mathbf{q}, \mathbf{y}^{\min}, \mathbf{y}^{\max}, \mathbf{t}^{\min}, \mathbf{t}^{\max}, \mathbf{p}^{\min}, \mathbf{p}^{\max}, \mathbf{w}^{\min}, \mathbf{w}^{\max}, \mathbf{r}, \rho, e, Q, \gamma$, and V_b . We will denote $\Pi_{J,T}$ the parameter space through which any instance $I \in \mathcal{I}$ can be uniquely determined by some parameter vector $\pi \in \Pi_{J,T}$ (given adequate J and T).

4. Graph Neural Networks (GNNs)

Graph neural networks are generalizations of convolutional neural networks from grid-structured data to graphs. The input to a GNN is a graph and a set of feature vectors associated with the graph nodes. GNNs work by propagating feature information between neighboring nodes, which can be seen as message-passing. More specifically, at each layer, the GNN first computes the messages sent by the nodes based on their feature vectors. Then, it updates each node's feature vectors based on the messages sent by their neighbors. We will refer to the feature update as a graph *convolution*, due to its similarities to the operations in convolutional neural networks.

At each layer $l \in \{1, \dots, L\}$, the graph convolution operation has two major components: *message functions* $M_l(\cdot)$, which compute the messages propagated by each node, and *update functions* $U_l(\cdot)$, which update the node-associated features based on the messages. Let $G = (V, E)$ be a graph and $\mathbf{h}_v^{(0)}$ be feature vectors for every node $v \in V$. The messages sent $\mathbf{m}_u^{(l)}$ are computed based on the node features

$$\mathbf{m}_u^{(l)} = M_l(\mathbf{h}_u^{(l-1)}), \forall u \in V, \tag{5}$$

where $M_l(\cdot)$ is the message function of layer l . After computing the messages, the features of node v are updated through

$$\mathbf{h}_v^{(l)} = U_l\left(\mathbf{h}_v^{(l-1)}, \text{Aggregation}(\{\mathbf{m}_u^{(l)} : u \in \mathcal{N}(v)\})\right), \tag{6}$$

where $\mathcal{N}(v)$ is the set of neighbors of v , $U_l(\cdot)$ is the update function of layer l , and **Aggregation** is a function that receives multiple message vectors and returns a single vector.

A common choice for graph convolutions is to use feedforward neural networks as message and update functions. Given that the number of neighbors varies, we need an **Aggregation**

function that always returns a fixed-length vector, invariant to input size, hence the naming. A natural choice is to add up the messages, but many are the possibilities, such as

$$\text{Aggregation} = \begin{cases} \frac{1}{|\mathcal{N}(v)|} \sum_{u \in \mathcal{N}(v)} \mathbf{m}_u^{(l)}, & \text{if mean} \\ \max_{u \in \mathcal{N}(v)} \mathbf{m}_u^{(l)}, & \text{if max} \\ \sum_{u \in \mathcal{N}(v)} \mathbf{m}_u^{(l)}, & \text{if sum} \\ \sum_{u \in \mathcal{N}(v)} \alpha_{u,v} \mathbf{m}_u^{(l)}, & \text{if attention} \end{cases}, \quad (7)$$

where $\alpha_{u,v}$ is an attention weight for node u given node v , which can be learned along the model parameters. Furthermore, the message function can easily be extended to consider the edge weight (or even edge features) along with the feature vectors of the neighbors.

A common approach is to define the message functions $M_l, l = 1, \dots, L$, as linear operators over the hidden features of the neighbors, aggregate these messages by summing, and use the ReLU (Rectifier Linear Unit) activation function with a bias as the update functions $U_l, l = 1, \dots, L$. An example of such an approach is the model proposed by Kipf and Welling (2017), which uses a linear combination of the features as the message function, aggregates messages by summation, and updates using a single-layer neural network with ReLU activation. More precisely, we can describe their convolution operation as

$$\begin{aligned} \mathbf{m}_{u,v}^{(l)} &= \frac{1}{c_{vu}} W^{(l)} \mathbf{h}_u^{(l-1)}, u \in \mathcal{N}(v) \\ \mathbf{h}_v^{(l)} &= \text{ReLU} \left(\mathbf{b}^{(l)} + \sum_{u \in \mathcal{N}(v)} \mathbf{m}_{u,v}^{(l)} \right) \end{aligned} \quad (8)$$

where $c_{vu} = \sqrt{|\mathcal{N}(u)|} \sqrt{|\mathcal{N}(v)|}$ with $|\mathcal{N}(v)|$ denoting the number of neighbors, and $W^{(l)} \in \mathbb{R}^{d_l \times d_l}, \mathbf{b}^{(l)} \in \mathbb{R}^{d_l}$ (layer l has size d_l) are the weights and biases of the model, i.e., learnable parameters.

Another convolution operator was proposed by Hamilton et al. (2017) and named SAGE (SAmple and aGgrEGate). The authors propose to directly aggregate the features of the neighbors, i.e., to use the identity as the message function and to use a single-layer network with ReLU activation as the update function. Putting it into terms, the proposed graph convolution is

$$\begin{aligned} \mathbf{m}_{u,v}^{(l)} &= \mathbf{h}_u^{(l-1)}, u \in \mathcal{N}(v) \\ \mathbf{h}_v^{(l)} &= \text{ReLU} \left(\mathbf{b}^{(l)} + W_1^{(l)} \mathbf{h}_v^{(l-1)} + W_2^{(l)} \text{Aggregation}(\mathbf{m}_{u,v}^{(l)}, u \in \mathcal{N}(v)) \right) \end{aligned} \quad (9)$$

where $W_1^{(l)}, W_2^{(l)} \in \mathbb{R}^{d_l \times d_l}, \mathbf{b}^{(l)} \in \mathbb{R}^{d_l}$ are the parameters. The authors suggest using more complex aggregation operators, such as an LSTM (Long Short-Term Memory network) and a fully connected single-layer neural network, followed by a pooling operation (element-wise

maximum). Note that the SAGE convolution with summation as the aggregation function is equivalent to the one proposed by Kipf and Welling.

After recurrent graph convolutions through the L layers of a GNN, we can use the resulting node features $H^{(L)} = \left[\mathbf{h}_v^{(L)} \right]_{v \in V}$ straight-away, which is suitable for, e.g., node classification tasks, or we can aggregate them further into a single feature vector, suitable for, e.g., graph classification tasks. The GNN can be trained end-to-end by minimizing a prediction loss based on its outputs, optimizing its parameters (e.g., $W^{(l)}$ and $\mathbf{b}^{(l)}$ of (8)) in the same way as a traditional deep learning model.

4.1. MILP Problems as Inputs for GNNs

We can feed MILP problem instances to GNNs by representing the relationship between problem variables and constraints through a bipartite graph (Gasse et al., 2019; Ding et al., 2020; Khalil et al., 2022; Nair et al., 2020; Han et al., 2023). More precisely, given a problem of the form

$$\begin{aligned} \max_{\mathbf{x}} \quad & \mathbf{c}^T \mathbf{x} \\ \text{s.t.:} \quad & A\mathbf{x} \leq \mathbf{b}, \end{aligned} \tag{10}$$

where $\mathbf{x} \in X \subseteq \mathbb{R}^n$, $A \in \mathbb{R}^{m \times n}$, $\mathbf{c} \in \mathbb{R}^n$, and $\mathbf{b} \in \mathbb{R}^m$, we can build a graph $G = (V_{\text{var}} \cup V_{\text{con}}, E)$, in which $|V_{\text{var}}| = n$, $|V_{\text{con}}| = m$, and $E = \{(v_{\text{con},i}, v_{\text{var},j}) : A_{i,j} \neq 0\}$. Intuitively, the graph represents the structure of the problem at hand, with edges capturing the relationship between variables and constraints. Note that this approach yields a bipartite graph, that is a graph in which the nodes are separated into two disjoint sets, V_{var} and V_{con} , with edges connecting only nodes from one set to the other.

To fully represent an MILP instance, however, the graph has to be enhanced with edge and node weights. Let $w : V \cup E \mapsto \mathbb{R}$ be the weight function¹. Then, for every node $v_i \in V_{\text{con}}$, associated with the i -th constraint, the weight will be the constraint's bound, i.e., i -th element of $\mathbf{b}$, i.e., $w(v_i) = b_i$. Similarly, for $v_j \in V_{\text{var}}$, associated with the j -th variable, $w(v_j) = c_j$, where $\mathbf{c} = (c_j)_{j=1,\dots,n}$. Edge weights are given by the incidence matrix A , that is, given $e = (v_{\text{con},i}, v_{\text{var},j}) \in E$, then $w(e) = A_{i,j}$. This graph representation approach establishes a bijective relationship to the instance space, i.e., every MILP instance is uniquely determined by a weighted graph and vice-versa².

In practice, the graph fed to a GNN is associated with feature vectors $\mathbf{h}_v^{(0)}, \forall v \in V$, of arbitrary size. In other words, the information contained in the features provided to the network is a design choice. It can contain the weights described above, but many other features might also help the model learn the graph-related task (Gasse et al., 2019; Nair et al., 2020).

¹Note that, in a slight abuse of notation, we use a single function to weigh both nodes and edges.

²For the weighted graph to identify an MILP instance uniquely, it would need to contain the information about the integrality of the variables. Restricting the bijection statement to MILP problems with the same number of integer variables would be more precise. Alternatively, to assume, without loss of generality, that the first $k \leq n$ variables (i.e., x_1 to x_k) are integer variables and use k as a weight for the whole graph.

For illustration purposes, consider an optimization problem in the form of Eq. (10) with three variables $\mathbf{x} = [x_1, x_2, x_3]^T$, three constraints C_1, C_2 and C_3 , and

$$\mathbf{c} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}; A = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & -1 \\ 5 & 0 & 1 \end{bmatrix}; \mathbf{b} = \begin{bmatrix} 2 \\ 1 \\ 4 \end{bmatrix}. \quad (11)$$

Then, a bipartite graph representation is illustrated in Figure 3. For the features, one may define

$$\mathbf{h}_{x_j}^{(0)} = \begin{bmatrix} c_j \\ \frac{1}{3} \sum_{i=1}^3 A_{ij} \\ \max_i A_{ij} \end{bmatrix} \quad \text{and} \quad \mathbf{h}_{C_i}^{(0)} = \begin{bmatrix} b_i \\ \frac{1}{3} \sum_{j=1}^3 A_{ij} \\ \max_j A_{ij} \end{bmatrix},$$

such that

$$H^{(0)} = \begin{bmatrix} \mathbf{h}_{x_1}^{(0)T} \\ \mathbf{h}_{x_2}^{(0)T} \\ \mathbf{h}_{x_3}^{(0)T} \\ \mathbf{h}_{C_1}^{(0)T} \\ \mathbf{h}_{C_2}^{(0)T} \\ \mathbf{h}_{C_3}^{(0)T} \end{bmatrix} = \begin{bmatrix} 1 & 2 & 5 \\ 2 & 1 & 2 \\ 3 & 0 & 1 \\ 2 & 1 & 2 \\ 1 & 0 & 1 \\ 4 & 2 & 5 \end{bmatrix}.$$

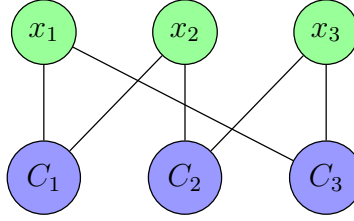


Figure 3: Bipartith graph representation of an MILP with three variables (x_1 , x_2 , and x_3), three constraints (C_1 , C_2 , and C_3), and A as in Eq. (11).

5. Methodology

This section presents our methodological approach to tackle the two research questions proposed in the introduction. As both questions concern learning tasks with GNNs on to the ONTS problem, we describe our approach to generating problem data used to train the deep learning models, and our proposed GNN architecture (SatGNN). Furthermore, and specifically for the experiments that tackle the second research question, we show our approach to embed trained deep learning models in heuristic solutions for the ONTS problem.

5.1. Data

High-quality data is necessary to learn the tasks of interest and perform reliable evaluations. To achieve a significant quantity of data, we generate new instances of the ONTS problem with variations of all task parameters and energy availability. However, randomly

sampling parameter values suitable for the problem formulation presented in Sec. 3.1 would not reflect the distribution of instances we expect to see in practice. Therefore, we use a methodological approach to sample the parameter values, following Rigo et al. (2023). The procedure for generating pseudo-random realistic parameters is detailed in Appendix B.

The algorithm for constructing our dataset is formalized in Algorithm 1. We optimize every new instance $I \in \mathcal{I}$ using a commercial solver with limited time, during which we collect the best candidate solutions found (represented by Z^*). Note that we collect only the values for the binary variables $\mathbf{z}$, as the binary ones can completely determine the continuous variables (see Sec. 3.1). The instance is rejected if no feasible solution is found during the time budget (or if the solver proves infeasibility).

Algorithm 1: Dataset generation. π is the parameter vector described in Sec. 3.1, Z is the set of all feasible solutions, and $Z^* \subset Z$ is the set of candidate solutions the solver finds. For a description of `RandomInstance` see Appendix B.

Data: Time horizon T , number of jobs J , number of instances (final dataset size) n .

Result: Dataset $\mathcal{D} = \{(I, Z^*) : Z^* \subset Z \text{ given } I\}$.

```

while  $|\mathcal{D}| < n$  do
     $\pi \leftarrow \text{RandomInstance}(T, J)$ 
     $I \leftarrow \text{BuildInstance}(\pi)$ 
     $Z^* \leftarrow \text{Solver}(I)$ 
    if  $|Z^*| > 0$  then
         $\mathcal{D}.\text{add}(I, Z^*)$ 
    end
end

```

5.2. SatGNN

We name *SatGNN* the base model proposed for all experiments. An overview of the components of our model can be seen in the diagram of Figure 4, and a summary of the symbols used is presented in Appendix C.

SatGNN is designed to take as input instances of the ONTS problem. Each instance is a mathematical program with a varying number of variables and constraints (i.e., varying number of jobs). Thus we expect our model to handle inputs of different sizes and to be invariant to the symmetries that mathematical programs exhibit (e.g., permutations of variables, constraints). For that reason, we follow the approach described in Section 4 of embedding each instance as a bipartite graph, and design SatGNN around a GNN (hence its name).

To fully represent each instance, the input graph is augmented with feature vectors for every node. Let $G = (V, E, w)$ be the graph representation of an instance $I \in \mathcal{I}$, as described in Sec. 4.1, with $V = V_{\text{var}} \cup V_{\text{con}}$ being the set of nodes and $w : E \rightarrow \mathbb{R}$ the edge-weight function based on the variable coefficients in the constraints. Let $\mathbf{f}_v$ denote the feature vector associated with node v . We add four features to constraint nodes and six to

variable nodes, as described in Table 2. We highlight that the features overdetermine the instance; that is, more features than necessary are provided to the model. This is common in deep learning practice and has been observed to yield positive results in combinatorial optimization applications (Nair et al., 2020; Khalil et al., 2022).

Table 2: Description of input features for SatGNN.

Features of constraint nodes ($\mathbf{f}_{v_{\text{con}}}$)	Features of variable nodes ($\mathbf{f}_{v_{\text{var}}}$)
Constraint’s upper bound ($\mathbf{b}$)	Variable’s coefficient in the objective ($\mathbf{c}$)
Constraint’s average coefficient (mean of A_{i*})	Variable’s average coefficient in the constraints (mean of A_{*j})
Number of neighbors/non-null coefficients ($ \mathcal{N}(v_{\text{con}}) $)	Number of neighbors/non-null coefficients ($ \mathcal{N}(v_{\text{var}}) $)
Whether it is an equality or an inequality constraint	Largest coefficient in the constraints ($\max(A_{*j})$)
	Smallest coefficient in the constraints ($\min(A_{*j})$)
	Whether it is a continuous or binary variable

The feature vectors are encoded into the first layer of hidden features by fully-connected neural networks. More precisely, the hidden feature vectors $H^{(0)} \in \mathbb{R}^{(n+m) \times d}$ (where n is the number of variables, m is the number of constraints, and d is the dimension of the hidden feature vectors) are computed by single-layer neural networks with ReLU activation

$$\begin{aligned} \text{NN}_{\text{var}} : \mathbb{R}^6 &\longrightarrow \mathbb{R}_+^d \\ \mathbf{f}_{v_{\text{var}}} &\longmapsto \mathbf{h}_{v_{\text{var}}}^{(0)} = \text{NN}_{\text{var}}(\mathbf{f}_{v_{\text{var}}}) \end{aligned}$$

and

$$\begin{aligned} \text{NN}_{\text{con}} : \mathbb{R}^4 &\longrightarrow \mathbb{R}_+^d \\ \mathbf{f}_{v_{\text{con}}} &\longmapsto \mathbf{h}_{v_{\text{con}}}^{(0)} = \text{NN}_{\text{con}}(\mathbf{f}_{v_{\text{con}}}), \end{aligned}$$

such that the initial hidden features associated to the variable nodes $H_{\text{var}}^{(0)} \in \mathbb{R}^{n \times d}$ are computed through NN_{var} and the initial hidden features associated to the constraint nodes $H_{\text{con}}^{(0)} \in \mathbb{R}^{m \times d}$ are computed through NN_{con} , in which $H^{(0)} = [H_{\text{var}}^{(0)}, H_{\text{con}}^{(0)}]$.

At each layer of SatGNN, the graph convolutions are performed in two steps, as in Gasse et al. (2019). First, a graph convolution is applied to update the hidden features of the constraint nodes. Then, another graph convolution is performed, this time considering the updated features of the constraint nodes to update the hidden features of the variable nodes. These two operations are illustrated in Figure 4 through the *GraphConv* blocks.

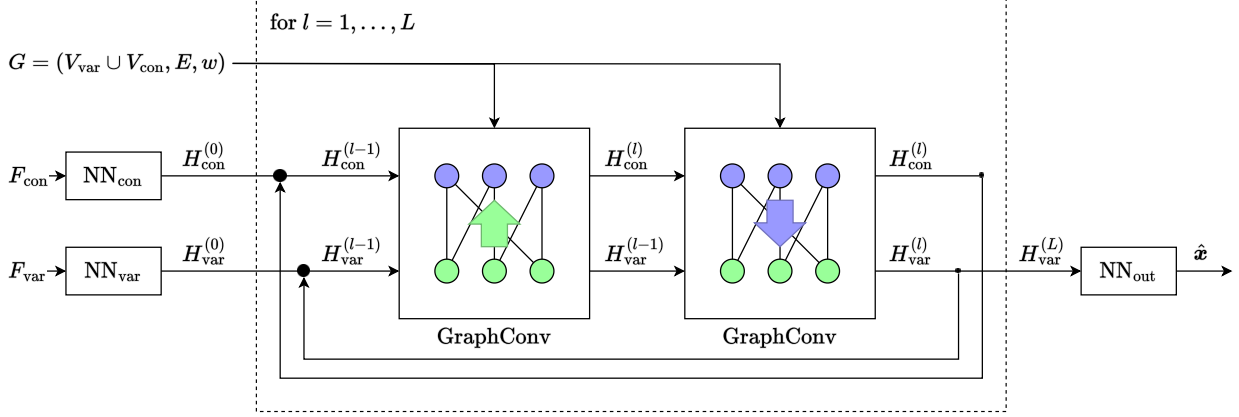


Figure 4: *SatGNN* architecture overview, where G is the bipartite graph representation of an MILP problem instance, H variables represent the sets of hidden features of the nodes, and NN. are neural networks. The connection of G and the *GraphConv* operators represents both the weights of the edges and the neighborhood information.

Finally, after L iterations, the last layer's hidden features associated to the variable nodes are fed to a neural network

$$\begin{aligned} \text{NN}_{\text{out}} : \mathbb{R}_+^d &\longrightarrow \mathbb{R} \\ \mathbf{h}_{v_{\text{var}}}^{(L)} &\longmapsto \hat{z}_{v_{\text{var}}} = \text{NN}_{\text{out}}(\mathbf{h}_{v_{\text{var}}}^{(L)}). \end{aligned}$$

This network combines the features extracted by the graph convolution operations into the desired shape of the output. The network contains two layers of d nodes with ReLU activation. For node classification tasks (Sec. 5.3.2), the output of the NN_{out} contains a sigmoid activation function, resulting in a probability prediction for each variable node. For graph classification tasks (Sec. 5.3.1), in which the output is a single probability value for the entire graph, the output of NN_{out} for every variable node is summed into a single value before the sigmoid.

5.3. Training

Two learning tasks are proposed to investigate the research questions that are the focus of this paper. To investigate whether a GNN can learn the structure of the ONTS problem (*first research question*), we train the *SatGNN* model on the feasibility classification of candidate solutions. To evaluate the effectiveness of a GNN-based heuristic (*second research question*), we train *SatGNN* to generate candidate solutions given instances of the problem.

5.3.1. Feasibility Classification

Given an instance $I \in \mathcal{I}$ of the ONTS problem and a candidate solution $\hat{\mathbf{z}}$, we aim for the model to classify whether $\hat{\mathbf{z}}$ is a feasible solution. We address this task using *SatGNN* by embedding the candidate solution as an additional feature in each variable node, thereby adding one extra dimension to every $\mathbf{f}_{v_{\text{var}}}$. Also, the model's output is a binary value (0 or 1), indicating whether the assignment is infeasible or feasible, respectively.

As briefly described in Section 5.2, the feasibility classification is a graph classification task. It is therefore natural to take advantage of maximum likelihood training. To this end, we define $y = p(\hat{\mathbf{z}} \in Z|I) \in \{0, 1\}$ as the target for each assignment and instance. Note that $p(\hat{\mathbf{z}} \in Z|I)$, which represents the probability that assignment $\hat{\mathbf{z}}$ is feasible, matches the label, once there is no stochastic component in the feasibility. We then use SatGNN to obtain an estimate $\hat{p}(\hat{\mathbf{z}} \in Z | I) \in [0, 1]$ of the true probability. In particular, we do so by aggregating the output of the last fully-connected module NN_{out} , and using the sigmoid function.

The model is trained to minimize the binary cross-entropy (BCE) between the predicted probability $\hat{p}(\hat{\mathbf{z}} \in Z|I) \in [0, 1]$ and the true probability $p(\hat{\mathbf{z}} \in Z|I) \in [0, 1]$. We note that the minimization is performed over the parameters of the graph convolutions, as well as the parameters of NN_{con} , NN_{var} , and NN_{out} .

The training data is built with instances from $\mathcal{D}$ (see Sec. 5.1). However, in each $(I, Z^*) \in \mathcal{D}$, we only have high-quality feasible solutions in Z^* , which is clearly not representative of the input space for the feasibility classification task. In other words, we need both feasible and infeasible candidate solutions for the model to learn the classification.

For that end, we augment Z^* with two other sets of candidate solutions. The first, which we call Z_R , is constructed from candidate solutions uniformly sampled from $\{0, 1\}^{2JT}$, forming a representative sample of the entire domain. The second set, Z_N , is constructed from candidate solutions in the neighborhood of Z^* . Our intuition behind Z_N is to increase the information density of our sample near the borders of the feasible region, which is where our target function is highly nonlinear. To achieve this, we build Z_N by randomly flipping a limited number of assignments in randomly selected candidate solutions from Z^* .

Finally, our dataset for feasibility classification can be described as

$$\mathcal{D}_{\text{feas}} = \{(I, \hat{\mathbf{z}}, y) \in \mathcal{I} \times Z \times \{0, 1\} : (I, Z^*) \in \mathcal{D}, \hat{\mathbf{z}} \in Z^* \cup Z_R \cup Z_N, y = p(\hat{\mathbf{z}} \in Z|I)\},$$

where $p(\hat{\mathbf{z}} \in Z|I)$ is computed for each candidate solution³ by evaluating the constraints of I .

5.3.2. Optimal Solution Prediction

We want SatGNN to predict an optimal solution given an instance of the ONTS problem. We expect this task to be more challenging than the previous one, as finding an optimal solution is significantly harder than checking feasibility. Furthermore, an instance may have multiple optimal solutions, so the target for the model is not trivially well-defined.

We formulate this task through the variable bias at optimality, that is, the probability that each binary variable takes a positive value in an optimal solution chosen at random under a uniform distribution. We denote this probability vector as

$$\mathbf{p}(\mathbf{z}^* = 1|I) = \begin{bmatrix} p(z_{1,1}^* = 1|I) \\ \vdots \\ p(z_{J,T}^* = 1|I) \end{bmatrix},$$

³Except for those in Z^* , which are always feasible.

where $p(z_{j,t}^* = 1|I)$ is the probability that the variable $z_{j,t}^*$ takes a positive value in an optimal solution to instance I . We use SatGNN to obtain an estimate $\hat{\mathbf{p}}(\mathbf{z}^* = 1|I)$, which we can sample to generate candidate solutions for I . In particular, we use thresholding to obtain a candidate solution.

The standard classification approach for this task is to train the model to minimize the BCE with respect to the best-known solution, which serves as the target. Note that the best-known solution is indeed the target when the model has a single optimal solution and the instance has been solved to optimality. For training with the *best solution*, we build the dataset by selecting the best $\mathbf{z}^* \in Z^*$ for each $(I, \mathbf{z}^*) \in \mathcal{D}$, i.e.,

$$\mathcal{D}_{\text{opt-b}} = \left\{ (I, \mathbf{z}^*) \in \mathcal{I} \times Z : (I, Z^*) \in \mathcal{D}, \mathbf{z}^* = \arg \max_{\hat{\mathbf{z}} \in Z^*} \text{QoS}(\hat{\mathbf{z}}; \mathbf{u}_I) \right\},$$

in which $\mathbf{u}_I$ is the vector of priorities from instance I (see Sec. 3.1).

Another training approach is the one proposed by Nair et al. (2020), in which multiple known solutions are used as targets, being weighted by their objective value. Note that the best-known solution is also used in this approach, alongside sub-optimal solutions. Intuitively, this *multi-target* approach guides the model toward the region of high-quality solutions, rather than solely toward the best-known solution, which may result in more feasible predictions.

The dataset for the multi-target approach simply requires computing the weights for each $\hat{\mathbf{z}} \in Z^*$ of each instance in $\mathcal{D}$. More specifically, we build

$$\mathcal{D}_{\text{opt-m}} = \left\{ (I, \hat{\mathbf{z}}, w_{\hat{\mathbf{z}}}) \in \mathcal{I} \times Z : (I, Z^*) \in \mathcal{D}, \hat{\mathbf{z}} \in Z^*, w_{\hat{\mathbf{z}}} = \frac{\exp(\text{QoS}(\hat{\mathbf{z}}; \mathbf{u}_I))}{\sum_{\mathbf{z} \in Z^*} \exp(\text{QoS}(\mathbf{z}; \mathbf{u}_I))} \right\}.$$

5.4. SatGNN-based Heuristic

Although we could use the SatGNN by itself as a heuristic for the ONTS problem by sampling its output to generate candidate solutions, it is unlikely that these solutions will be useful. Because our problem has constraints, an error in a single variable may result in an infeasible candidate. Given that ONTS problems of moderate size already have thousands of variables⁴, infeasibility is likely to occur. To circumvent this limitation, we use the deep learning model in conjunction with a mathematical programming solver as a primal heuristic. In particular, we propose three approaches for doing so, which instantiate recent GNN-guided baselines (Nair et al., 2020; Han et al., 2023).

5.4.1. Warm-starting

A straightforward way to aid most commercial-grade solvers is to warm-start them with good candidate solutions. For example, the SCIP solver (Bestuzheva et al., 2021) accepts complete and partial solutions to the problem, which are used to guide the inner heuristics

⁴The smallest instance evaluated by Rigo et al. (2022) has 1746 binary variables, considering both x and ϕ variables.

during the optimization process. We use the output of the model to determine which variables will compose the partial solution provided to the solver based on the *confidence* of the model's prediction. In other words, we warm-start the solver with partial solutions built using only the assignments for variables for which the model predicts values closest to 0 or 1.

Formally, let $\hat{\mathbf{p}}(\mathbf{z}^* = 1|I)$ be the output of SatGNN given an instance $I \in \mathcal{I}$ as defined in Sec. 5.3.2. Our definition of confidence for a given variable comes from the maximum predicted marginal probability. For instance, the confidence with respect to variable $x_{j,t}$ of an instance I is

$$\kappa(x_{j,t}|I) := \begin{cases} \hat{p}(x_{j,t}^* = 1|I), & \text{if } \hat{p}(x_{j,t}^* = 1|I) \geq 0.5 \\ \hat{p}(x_{j,t}^* = 0|I) = 1 - \hat{p}(x_{j,t}^* = 1|I), & \text{otherwise.} \end{cases}$$

The confidence associated with the $\phi_{j,t}$ variables can be described similarly. Note that the confidence is always between 0.5 and 1. Furthermore, we write $\boldsymbol{\kappa}(\mathbf{z}^*|I)$ to denote the confidence vector over all variables.

Let us write $\mathbf{z} = (\mathbf{x}, \boldsymbol{\phi}) = (z_1, \dots, z_k, \dots, z_{2JT})$. We define $\mathcal{C}_I^N \subseteq \{1, \dots, 2JT\}$ as the set of indices of the N variables that the model is most confident of, that is, $|\mathcal{C}_I^N| = N$ and

$$\mathcal{C}_I^N = \{k \in \{1, \dots, 2JT\} : \kappa(z_k^*|I) \geq \kappa(z_{k'}^*|I), \forall k' \notin \mathcal{C}_I^N\}.$$

Then, a partial solution $\bar{\mathbf{z}}^{(N)}$ of size N can be written

$$\bar{\mathbf{z}}^{(N)} = [\hat{z}_k]_{k \in \mathcal{C}_I^N},$$

which contains the predicted assignments of the highest confidence and can be provided to the solver.

Note that warm-starting does *not* configure a heuristic solution to the problem, as it just modifies the behavior of the heuristics already present in the algorithmic solution. Therefore, we maintain optimality and feasibility guarantees, even in the worst-case scenario when the model provides the solver with bad, infeasible solutions.

5.4.2. Early-fixing

Our second matheuristic is similar to the Neural Diving approach proposed by Nair et al. (2020). Instead of merely informing the solver of a partial candidate solution via warm-starting, we modify the instance of the problem and fix the values of the variables that the model is most confident of. More precisely, early-fixing is equivalent to adding constraints of the form

$$z_k = \hat{z}_k, \forall k \in \mathcal{C}_I^N$$

to the mathematical program of instance I , limiting N variables to the assignments predicted by the model. In practice, however, no constraints are added, but the variables are replaced by the assignment in the model formulation. This is equivalent to a diving heuristic with respect to the branch-and-bound tree of instance I . The resulting sub-problem will contain fewer binary variables and, thus, is expected to be faster to solve.

We highlight, however, that while Nair et al. (2020) sample multiple partial solutions from the model output and solve all of them, we sample only one, and with a fixed size. This design choice is based on the fixed time-budget available for the ONTS problem at hand (see Section 3). With a single partial solution of fixed size, we have more control over the amount of binary variables in the program fed to the solver, which correlates to the solving time. In other words, we trade solution quality, which would be expected to be higher if more partial solutions were evaluated, for lower computational cost, since a single problem is solved.

Note that every feasible solution to the early-fixed problem and the fixed variables is also feasible in the original instance. However, an optimal solution to the sub-problem might not be optimal for the original instance. Even worse, early-fixing the problem might render it infeasible if the model’s output result in an infeasible partial assignment. Therefore, this approach of early-fixing is a matheuristic, as it uses mathematical optimization to find heuristic solutions.

5.4.3. Trust Region

The third approach follows the learning-based heuristic proposed by Han et al. (2023). Instead of fixing the variables that the model is most confident of, we allow a limited deviation from the sampled partial candidate solution. In other words, we define a *trust region* around the candidate solution.

In practice, a new constraint is added to the mathematical programming formulation of each instance I that limits feasible solutions to a neighborhood around the partial solution predicted by SatGNN. The constraint has the form

$$\sum_{k \in \mathcal{C}_I^N} |z_k - \hat{z}_k| \leq \Delta,$$

where $\Delta \in \mathbb{N}$ is the size of the trust region.

Note that we do not make any distinctions between 1 and 0 assignments, as done in Han et al. (2023). We follow the same confidence-based approach as in the previously proposed methods to select the variables. We do so because there is no *a priori* bias in optimization problems toward 1 or 0 values. Furthermore, even if the instances in $\mathcal{I}$ exhibit such a bias—as is indeed the case for the ONTS problem—we expect the model to learn it during training, so that it is reflected in the predicted marginal probabilities (and, consequently, in the confidence score).

In the above definition, it is easy to see that the early-fix is a particular case of the trust region method with $\Delta = 0$. In fact, just like the early-fixing approach, allowing a trust region around the (partial) candidate solution also configures a matheuristic, as neither optimality nor feasibility are guaranteed.

6. Experiments and Results

The experiments for this paper were conducted in Python, using PyTorch, the DGL libraries, and the SCIP solver, on a server with an Intel i7-12700 16-Core (12 cores, 20

threads), an NVIDIA RTX A4000, 16 GB of memory, running Ubuntu 22.04.1 LTS 64 bits. All the code written for our experiments, as well as detailed instructions on how to reproduce our results, are available in the paper’s accompanying repository⁵.

In the following sections, we present three experiments addressing the two research questions raised in the introduction (Sec. 1). In the first experiment, we train the proposed SatGNN model to classify the feasibility of candidate solutions for problem instances. We consider three scenarios of increasing difficulty with respect to the generalization performance. In the second experiment we train SatGNN models to predict the bias for the binary variables of problem instances, effectively learning the probability that each binary variable has to assume a positive value in an optimal solution. We compare two training approaches, considering either the optimal solution as the target or multiple feasible solutions as targets for each instance. Finally, we use the models trained for optimality prediction to generate candidate solutions, which are used to construct heuristics to the ONTS problem. More specifically, we use the trained models to construct three approaches (as presented in Sec. 5.4) to improve the SCIP solver in its off-the-shelf setting: warm-starting, early-fixing, and trust region.

6.1. Dataset

The data for the following experiments is generated according to the procedure described in Sec. 5.1. The dataset construction requires verifying feasibility solving to (quasi-)optimality instances of the ONTS problem. In other words, the dataset construction imposes a high computational cost, which scales with the size of the problem.

To circumvent the cost of dataset generation, we build a training set with many smaller instances and validation and test sets with fewer but larger instances. This way, we can afford a sufficient volume of data for training while evaluating at the most relevant instances, which have a higher solving cost. At the same time, we test the model’s generalization capability by evaluating its performance on instances harder than those seen during training.

We base our data generation in the instances proposed by Rigo et al. (2023), which take the FloripaSat-I mission as a reference (Marcelino et al., 2020) (see Appendix B). More precisely, we focus on instances with scheduling horizon $T = 125$.

The dataset is generated through Algorithm 1. We solve each instance and gather 500 of the best solutions possible, limited to a time budget of 5 minutes. More precisely, every instance $(I, Z^*) \in \mathcal{D}$ is such that $|Z^*| = 500$. An example of the optimal schedule for one of the generated instances with $J = 9$ can be seen in Figure 1. All instances and solutions are made publicly available (Pacheco et al., 2023b).

We generate 200 instances for each $J \in \{9, 13, 18\}$ jobs, and 40 instances for each $J \in \{20, 22, 24\}$ jobs, in a total of 720 instances. A summary of the size (in terms of variables and constraints) of each instance in our dataset is seen in Table 3. We underscore the difference in difficulty between the instances in the training set and the ones in the validation and test sets, as evidenced by the significant increase in the average primal-dual gap and decrease in

⁵<https://github.com/brunompacheco/sat-gnn>

the lower bound as measured by the relative objective value, when solving those instances using our baseline method (SCIP) within the time limit.

Table 3: Summary of instances in our dataset. Columns m and n indicate the number of constraints and variables, resp., in each instance. Under *Dataset*, we present the number of instances in each part of our dataset. Columns in *Baseline optimization results* have summary statistics after using our baseline optimization method (SCIP) with the time budget (T). More specifically, *Feasible* and *Optimal* indicate, respectively, the fraction of instances that were solved to feasibility and optimality under the time budget, *Gap* is the primal-dual gap, and *LB* is the average relative objective of the candidate solutions.

# of jobs	m	n	Dataset			Baseline optimization results			
			Train	Val.	Test	Feasible	Optimal	Gap	LB
9	7518	2250	200			98%	20%	0.03	98%
13	10618	3250	200			96%	3%	0.05	95%
18	13434	4500	200			85%	0%	0.09	83%
20	14633	5000		20	20	78%	0%	0.13	75%
22	16430	5500		20	20	60%	0%	0.17	56%
24	18664	6000		20	20	48%	0%	0.31	42%

6.2. Feasibility Classification

As described in Sec. 5.3.1, we build the dataset for feasibility classification $\mathcal{D}_{\text{feas}}$ by adding random candidate solutions to Z^* for every $(I, Z^*) \in \mathcal{D}$. More precisely, for every instance of the ONTS problem, we generate Z_R and Z_N such that $|Z_R| = 250$ and $|Z_N| = 250$, in a total of 1000 solutions for every instance.

SatGNN is modified for graph classification as described in Sec. 5.3.1. We use the graph convolution proposed by Kipf and Welling (2017), as presented in equation (8), and a single layer ($L = 1$). The hidden features all have a fixed size $d = 8$. Both NN_{con} and NN_{var} are neural networks with a single layer and ReLU activation, while NN_{out} is an artificial neural network with 3 layers and ReLU activation in the hidden layers and sigmoid activation at the last layer. For all feasibility classification experiments, SatGNN is trained using Adam (Kingma and Ba, 2015) with a learning rate of 10^{-3} to minimize the binary cross-entropy between the prediction and the feasibility of the candidate solution. The model is trained until the training loss becomes smaller than 10^{-2} , limited to a maximum of 200 epochs.

First, SatGNN is trained for one particular instance. Given an instance I , we build $\mathcal{D}_{\text{feas}}^I = \{(\hat{z}, y) \in Z \times \{0, 1\} : (I, \hat{z}, y) \in \mathcal{D}_{\text{feas}}\}$. This dataset is randomly divided into a training and a test set in an 80-20 split, so 800 instances are used to train the model. For each number of jobs $J \in \{9, 13, 18, 20, 22, 24\}$, we select 5 different instances randomly. In all experiments (all instances of all sizes), *SatGNN achieves 100% accuracy*, that is, the model is able to perfectly distinguish between feasible and infeasible candidate solutions in the test datasets.

Then, SatGNN is trained to generalize across instances with the same number of jobs. For each number of jobs J , we build the dataset by selecting only instances (and the respective

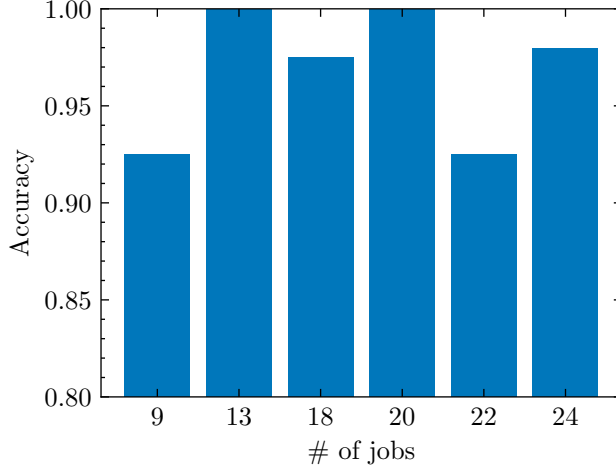


Figure 5: Test set performance of SatGNN trained for feasibility classification of candidate solutions given a fixed number of jobs. All instances used for testing were not seen by the model beforehand.

candidate solutions) with the same number of jobs, i.e.,

$$\mathcal{D}_{\text{feas}}^J = \{(I, \hat{z}, y) \in \mathcal{D}_{\text{feas}} : J_I = J\}, \quad (12)$$

where J_I represents the number of jobs of instance I . 20 of the generated instances are randomly selected for training, in a total of 20,000 training samples, while 10 different instances are randomly selected for testing. SatGNN achieves a very high performance, with over 90% accuracy on all settings. The complete test set performance for the different number of jobs can be seen in Figure 5.

Finally, SatGNN is trained solely on candidate solutions from small instances (i.e., those with $J \in \{9, 13, 18\}$) and tested on candidate solutions from large instances (i.e., those with $J \in \{20, 22, 24\}$). Following the notation of equation (12), we use $\mathcal{D}_{\text{feas}}^9 \cup \mathcal{D}_{\text{feas}}^{13} \cup \mathcal{D}_{\text{feas}}^{18}$ for training and $\mathcal{D}_{\text{feas}}^{20} \cup \mathcal{D}_{\text{feas}}^{22} \cup \mathcal{D}_{\text{feas}}^{24}$ for testing. In total, 60,000 samples are used for training and 30,000 samples are used for testing. SatGNN achieves 94.15% accuracy in the test set. The performance across the different sizes of instances and the groups of candidate solutions is presented in Table 4. We see that the model has more difficulty in the candidate solutions from the Z_N sets, the candidate solutions in the edge of the feasible region. It is also true that the performance decreases with problem size (number of jobs), indicating an increasing difficulty, as expected.

To further assess the architecture of the SatGNN model, we perform an ablation study on the number of layers in the NN_{out} component. We repeat the training and evaluation of the SatGNN model in the feasibility classification task across instance sizes for different numbers of layers in the last MLP of our model. More specifically, we experiment with up to 4 layers in NN_{out} , and for each number of layers, we repeat the experiment 5 times with different random initializations. The average performance of those models appears in Figure 6. No significant difference can be observed between the experiments, indicating that the learning primarily occurs before NN_{out} , in the graph convolutional layers. In fact,

Table 4: Feasibility classification performance of SatGNN on instances larger than those used for training. The test set is discriminated into the three sets of candidate solutions that compose it (see Sec. 5.3.1). Each row indicates the set of 10 instances with the same size (number of jobs).

# of jobs	Accuracy			
	Z^*	Z_N	Z_R	Total
20	100%	90%	100%	97.5%
22	100%	80%	100%	95%
24	90%	90%	89.84%	89.96%

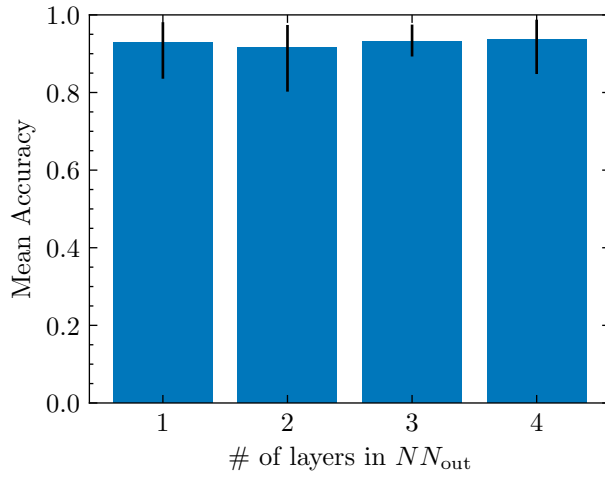


Figure 6: Test set performance of SatGNN with a varying number of layers in NN_{out} . Accuracy is measured as an average over 5 random repetitions of the experiment. Vertical bars indicate the minimum and maximum values observed.

applying the Wilcoxon signed-rank test (Wilcoxon, 1945) shows no statistically significant ($p\text{-value} > 0.05$) difference between the samples; however, this result should be taken with caution in the face of the size of the samples.

6.3. Optimal Solution Prediction

Following the methodology presented in Sec. 5.3.2, we first generate the datasets $\mathcal{D}_{opt-b}$ and $\mathcal{D}_{opt-m}$ from the dataset $\mathcal{D}$ (described in Sec. 6.1). Having both datasets allows us to compare the two different training strategies: training with a single target (quasi-optimal solution), and training with the best solutions found (which include the quasi-optimal solutions). The data is divided into training, validation, and test sets. The training sets contain the 200 small instances from $\mathcal{D}$, with 9, 13 or 18 jobs, while the validation and test sets contain, each, 20 large instances from $\mathcal{D}$, with 20, 22 or 24 jobs, such that no instance is used in both validation and test (empty intersection).

The models are trained according to the description provided in Sec. 5.3.2. Both when training with the best solution for each instance and when training with multiple solutions,

the models are trained using Adam to minimize the BCE between the prediction and the targets.

We tune the hyperparameters of the models by performing a random search and evaluating the models’ performance on the validation set. More specifically, we optimize the learning rate, number of convolutional layers (L), number of hidden features (d), graph convolutional operator, and whether or not the convolutions share the parameters. Table 5 shows the ranges and best values found. We provide further details on the hyperparameter tuning, including a sensitivity analysis, in Appendix D. Nonetheless, we underscore that the most impactful hyperparameters were the learning rate and the choice of graph convolution. In particular, the choice of the SAGE convolutional operator was strongly correlated with lower validation losses for both the models trained with the best solution and those trained with multiple targets.

Table 5: Hyperparameter space and best values for both models trained for optimality prediction. *BS* and *MS* indicate, resp., the models trained with the best solution and multiple solutions as targets. *Weight tying* refers to the parameter sharing process between the two graph convolutional operators in the model (see Section 5.2).

Hyperparameter	Search space	Best value	
		BS	MS
Learning rate	$\{10^{-2}, 10^{-3}, 10^{-4}\}$	10^{-2}	10^{-3}
Layers (L)	$\{1, 2, 3\}$	2	3
Hidden features (d)	$\{2^5, 2^6, 2^7, 2^8\}$	2^6	2^8
Graph convolution	{GraphConv, SAGE}	SAGE	SAGE
Weight tying	{Yes, No}	Yes	Yes

Our training stopping criteria is determined through the validation set, i.e., from all epochs of our training budget (100 epochs), we pick the model that achieves the lowest validation loss. The training curve for both training approaches can be seen in Figure 7. After training with the best solution, the average BCE on the validation set is 0.2887 and on the test set 0.2873. After training with multiple solutions, the average BCE on the validation set is 0.2451 and on the test set 0.2482. Although the BCE values are not comparable across training approaches, the small difference between the validation and the test sets for both approaches indicates no sign of overfitting.

To grasp the actual performance of the models in the optimality prediction, we evaluate both on the test set after training. Because of the high number of variables, neither could predict all variable assignments of any instance perfectly. However, when considering each variable assignment individually, the model trained with multiple solutions and the model trained with the best solutions achieved 90% and 88% average accuracy, respectively. We point out that the data is naturally biased towards zero assignments, as most jobs pass most of the time inactive (not assigned to run) when there are many jobs to be scheduled. Given the imbalance of the data, the performance of both models is better visualized in Figure 8. Furthermore, the model trained with multiple solutions achieved an F1 score of 0.72, while

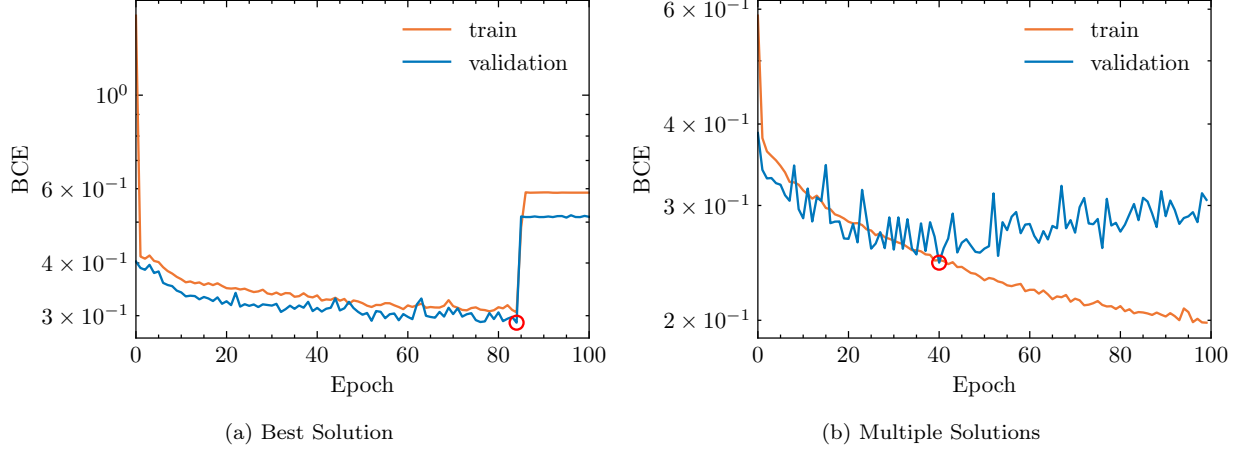


Figure 7: Training curves for SatGNN models trained with (a) the best solution or (b) multiple solutions. The best average BCE on the validation set (highlighted in red) is used for early-stopping the training.

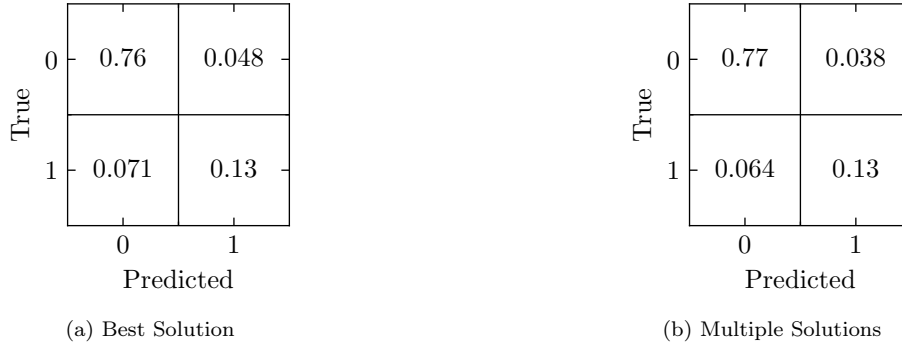


Figure 8: Accuracy of the models when predicting optimal assignments. Computed with respect to all variables of all instances in the test set.

the model trained with the best solutions achieved an F1 score of 0.68.

For building useful primal heuristics, the models should achieve consistent behavior across instances, meaning they should correctly predict a high number of variable assignments. Under that perspective, we measure the accuracy within each instance, which is calculated as the ratio of correctly classified variable assignments within a given instance. The performance is detailed in the histogram of Figure 9. We see that both models achieve high expected accuracy values, and none of them ever correctly predicts less than 75% of the variables correctly.

A deeper analysis is performed on the models' confidence when predicting variable assignments $\kappa(\mathbf{z} = 1|I)$, as presented in Sec. 5.4.1. The average confidence of both models on instances of the test set is depicted in Figure 10 with respect to the binary variables of the optimization problem. Training with multiple solutions seems to result in a more confident model overall. Furthermore, both models provide significantly more confident predictions for the ϕ variables.

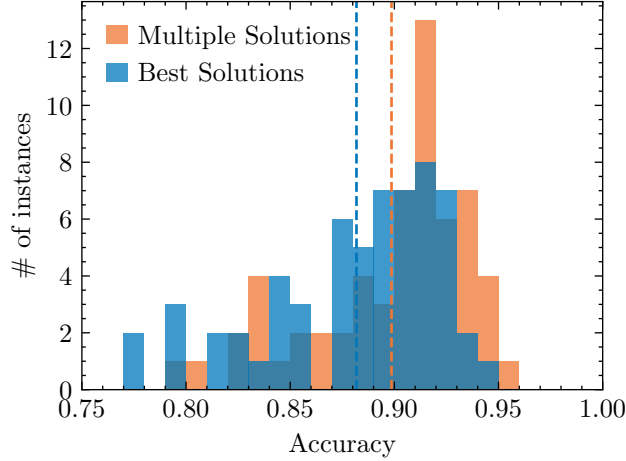


Figure 9: Histogram of accuracies for both models over the test set. For each instance, we compute the ratio of correctly predicted variable assignments. Vertical dashed lines indicate the mean value.

6.4. SatGNN-based Heuristic

The two models presented in Sec. 6.3 are used (each) to build three matheuristics, as described in Sec. 5.3.2. Namely, the two models are the SatGNN trained with the best solution available and the SatGNN trained with multiple solutions (see Sec. 5.3.2). The three matheuristics evaluated are: warm-starting, early-fixing, and trust region. We use SCIP as our solver both for the baseline results and the optimization within the matheuristics.

Although all model parameters and hyperparameters are already defined (and will not change) by the time the heuristics are assembled, the heuristics have their own hyperparameters, which require adjustment. More specifically, all three heuristic approaches have the hyperparameter $N \in \mathbb{N}$, representing the size of the partial solution extracted from SatGNN. Besides, the trust region matheuristic has the size of the trust region $\Delta \in \mathbb{N}$.

We use the validation set to select the best values for the hyperparameters, aiming to optimize two performance metrics. For once, we select values for the hyperparameters that maximize the expected objective value. The objective values are normalized, i.e., we divide them by the objective value of the best solution known for each instance⁶, resulting in a value between 0 (no QoS) and 1 (max. QoS known for the instance). In parallel, we select the hyperparameters that minimize the time required to find a feasible solution to each problem. The two tunings are performed for the SatGNN model trained with the best solution available and the SatGNN trained with multiple solutions. Table 6 summarizes the best hyperparameters for each model and each heuristic approach, in both evaluations.

The heuristics are then evaluated on the test set, which is not used for tuning neither the deep learning models' hyperparameters nor the heuristics' hyperparameters. The final performance is measured through the expected relative objective value given the time budget,

⁶We remind the reader that those solutions were computed along with the dataset generation, as detailed in Section 6.1.

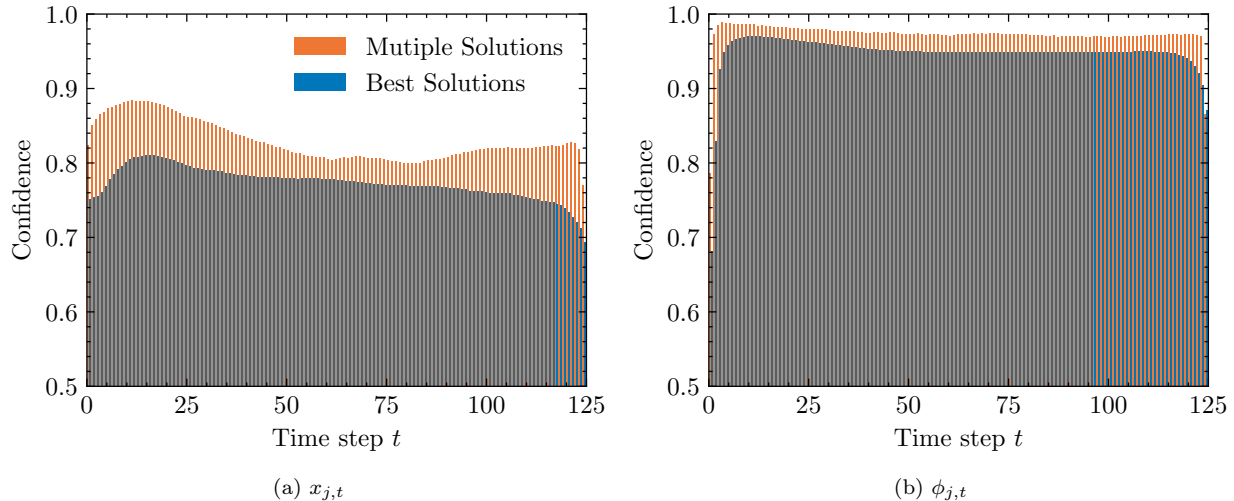
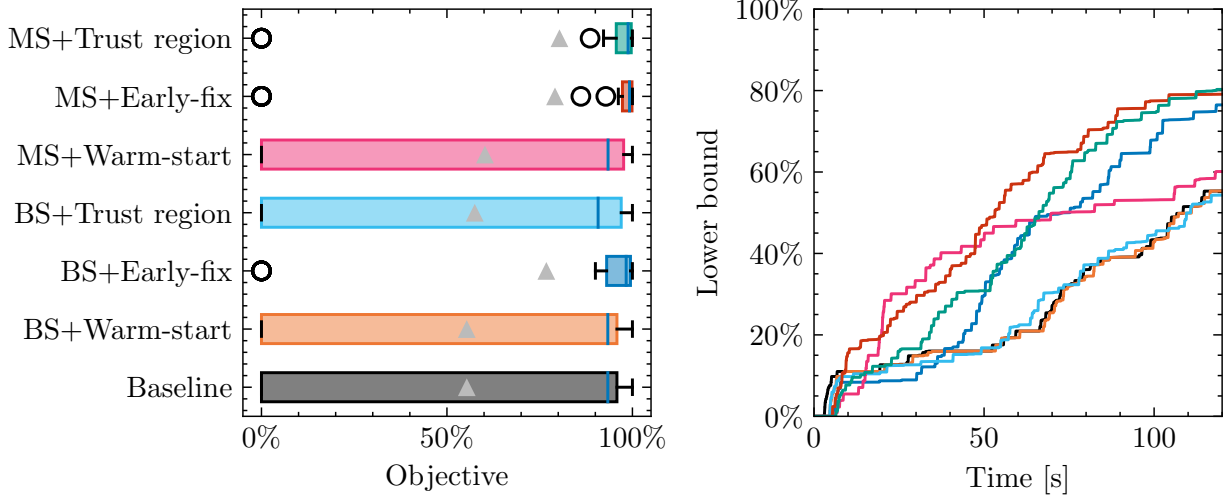


Figure 10: Average confidence (probability of the predicted class) of predicted values for (a) $\mathbf{x}$ and (b) ϕ variables of the model trained only with the best solution and the model trained with multiple solutions. The average is taken over all jobs of all instances in the test set.

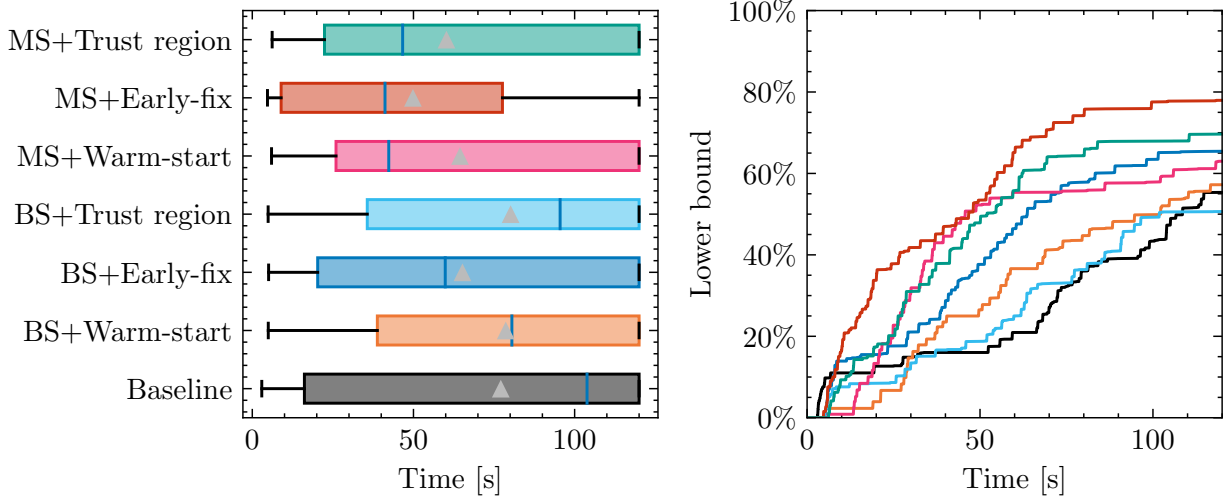
and the time it takes the heuristics to find a feasible solution. Furthermore, we also analyze the progress of the lower bound over time, indicating the expected performance under more restricted budgets. Figure 11 illustrates the results.

To assess the significance of the results presented in Figure 11 through statistical tests. Given that the matheuristic performance data is generated from a shared set of instances, our samples are paired. Furthermore, there is no evidence of normality in the results' distribution. Therefore, we apply the Wilcoxon signed-rank test (Wilcoxon, 1945), which is a non-parametric version of the t-test for matched pairs. We apply the test pair-wise, comparing each matheuristic approach to every other matheuristic approach, and to the baseline. The results are summarized in the critical difference diagram of Figure 12. Through early-fixing, both SatGNN models provide statistically significant ($p\text{-value} > 0.05$) improvements over the baseline in both performance metrics. However, the results show a significant advantage of the model trained with multiple solutions, providing better solutions (objective value) through early-fixing and trust region, and speeding up the feasibility definition (time to find a feasible solution) through all heuristic approaches.

Through the SatGNN model trained with multiple solutions, a 43% increase in the expected objective value within the time budget is achieved by early-fixing the partial solution, while defining a trust region around that same partial solution resulted in an expected 45% increase. Although the results are close, we see from the progress of the lower bound (in Fig. 11a, the plot on the right) that the early-fix heuristic is able to find better solutions more quickly than the trust region method during the time budget. In terms of feasibility, the early-fixing strategy using the same SatGNN model reduces in 35% the expected time to find a feasible solution, having a significant advantage over all other heuristic approaches. Surprisingly, even when optimized (through hyperparameter tuning) for reducing the time to find a feasible solution, the early-fix heuristic still improves the expected objective value

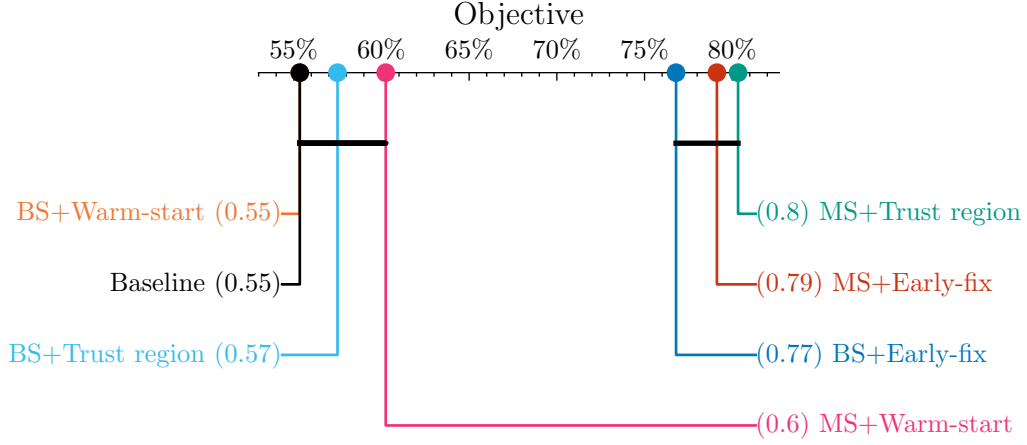


(a) Best solution found within the time budget.

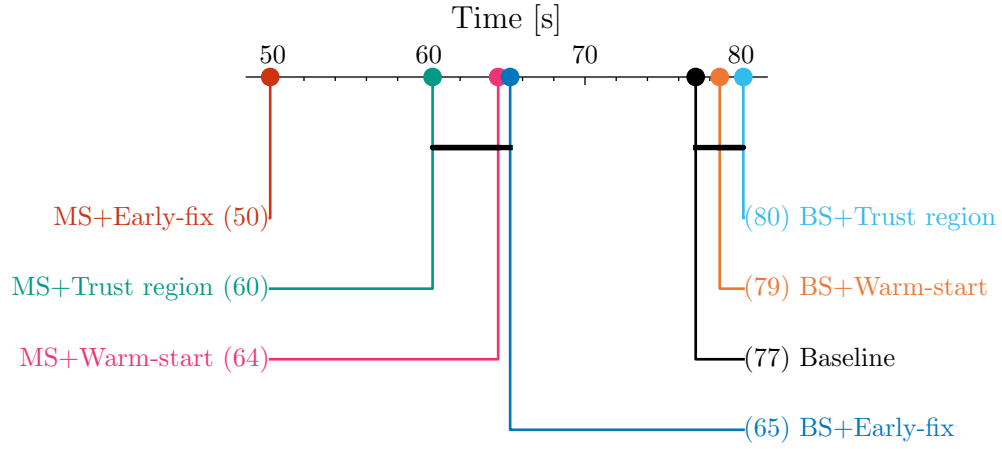


(b) Time to find a feasible solution.

Figure 11: Test set performance of the SatGNN-based matheuristics. On the left, we have the distribution of the evaluation metric of interest over the instances of the test set for the multiple approaches, in which the triangle indicates the mean value and the circles indicate outliers. *MS* indicates that the SatGNN model trained with multiple solutions was used, whereas *BS* indicates that the model trained solely with the optimal solution was used instead. On the right is the average progress of the lower bound on all test set instances. The objective value is considered relative to the objective of the best solution known, thereby always lying in the unit interval. The heuristics' hyperparameters N and Δ are defined upon experiments on the validation set, as presented in Table 6.



(a) Best solution found within the time budget.



(b) Time to find a feasible solution.

Figure 12: Critical difference diagram presenting the average test set performance of the SatGNN-based matheuristics (round marker in the axis). A crossbar between two (or more) approaches indicates that their performance (distribution on the test set) was not deemed significantly different ($p\text{-value} > 0.05$) through the paired Wilcoxon signed-rank test (Wilcoxon, 1945).

Table 6: Best values for partial solution size (N) and trust region size (Δ) for each heuristic approach and each SatGNN model, as evaluated on the validation set. *Objective* corresponds to the values that maximize the objective value of the best solution found. *Feasibility* indicates the values that minimize the amount of time to find a feasible solution.

SatGNN model	Heuristic	Objective		Feasibility	
		N	Δ	N	Δ
Best Solution	Warm-start	750	-	1000	-
	Early-fix	500	-	750	-
	Trust region	1000	5	1000	1
Multiple Solutions	Warm-start	1750	-	1500	-
	Early-fix	1000	-	1250	-
	Trust region	1250	1	1750	1

of the best solution found in 41%, as the lower bound progress illustrates (in Fig. 11b, the plot on the right).

6.4.1. Sensitivity Analysis

The proposed solution methods share a common parameter: the number N of variables in the partial solution. To evaluate the sensitivity of the results to this parameter, we evaluate the SatGNN model on the test set for varying values of N . More specifically, we vary the value of N for all matheuristics and measure the impact on the test set considering both perspectives (objective value and time to find a feasible solution). For conciseness, we consider only the SatGNN model trained with multiple solutions, as it is overall superior, in our experiments, to the model trained solely with the best solution. The results are illustrated in Figure 13.

Beyond N , the trust region method has an additional parameter: the size of the trust region, Δ . To evaluate the full potential of this matheuristic, we also consider three different values for Δ in our sensitivity analysis experiments. These results are also displayed in Figure 13, as the legends indicate.

As expected, the number of variables that compose the partial solution is impactful for all approaches, with early-fixing being the one most sensitive to it, and warm-start the less sensitive one. In fact, analyzing the results for the early-fixing approach in detail, we note that N indeed imposes a trade-off between solution quality and speed, as shown in Figure 14, confirming the intuitive interpretation of this parameter discussed in Section 5.4.2. More specifically, increasing the partial solution size seems to speed up the solver, allowing it to find better solutions more quickly, while reducing the final solution quality at the end of the time budget.

Furthermore, we note a relation between N and Δ through the peak performance of the trust region method (including early-fixing, which is equivalent to trust region with $\Delta = 0$). In other words, larger partial solutions seem to require larger trust regions to achieve peak performance. Intuitively, this result is expected since the partial solutions are based on

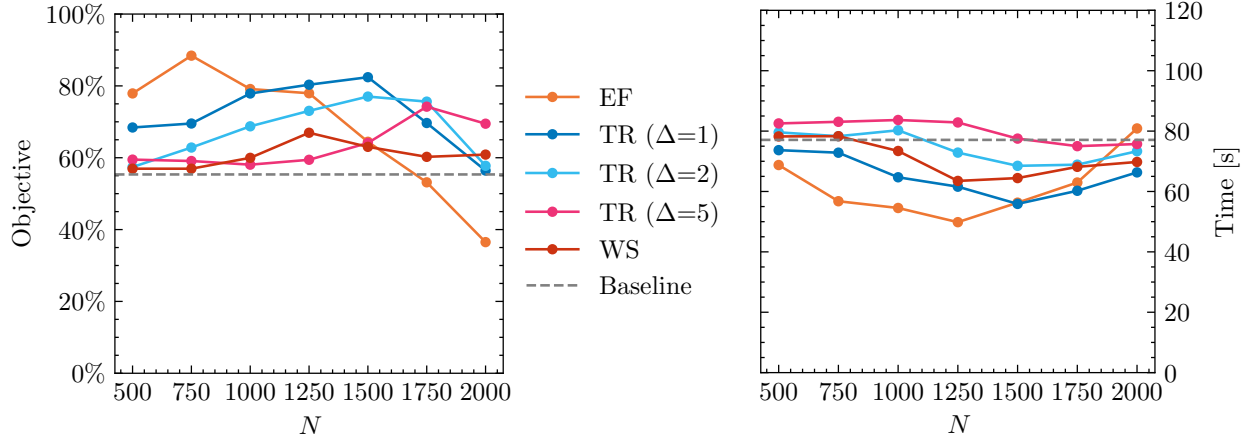


Figure 13: Sensitivity analysis of the solution methods with respect to the size of the partial solution N . *EF* indicates the early-fixing matheuristic, *TR* the trust region, and *WS* warm-start. The SatGNN model trained with multiple solutions is used. The evaluation is performed on the test set.

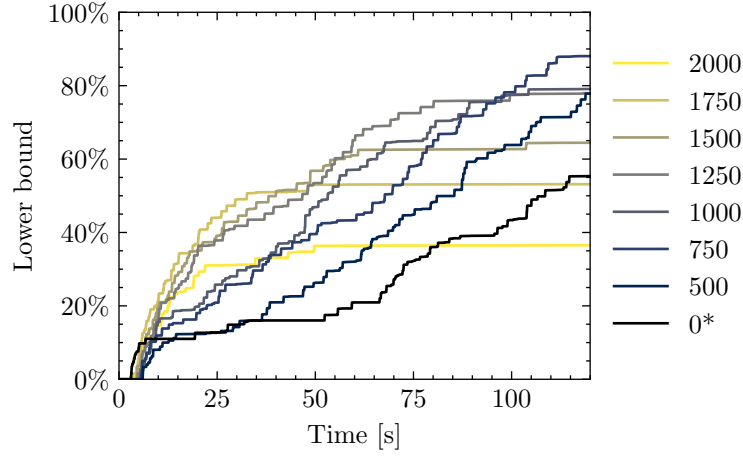


Figure 14: Average progress of the lower bound of SatGNN-based early fixing on the test set for varying values of the partial solution size N .

confidence, and, thus, the larger the partial solution is, the higher the expected probability of it including wrongly predicted variables. In turn, it suggests that the SatGNN model is properly trained (its confidence correlates to its performance).

7. Discussion

In this section, we discuss the results obtained from a series of experiments to address the research questions posed in the introduction. The experiments aimed to evaluate the effectiveness of GNNs in solving the ONTS problem. More specifically, we use our proposed model, SatGNN, based on the models by Gasse et al. (2019) and Hamilton et al. (2017). Our experiments evaluate whether SatGNN can learn feasibility and optimality given large instances of the ONTS problem and whether the trained model can be used in heuristic solutions.

The results of the first set of experiments (Sec. 6.2) indicate that SatGNN can provide accurate feasibility estimations even in challenging scenarios. Three scenarios of increasing difficulty were considered, and the results were quite promising, with over 89% accuracy in all experiments. In the most challenging scenario, SatGNN is evaluated on large ($J \in \{20, 22, 24\}$) unseen instances, but trained on small instances ($J \in \{9, 13, 18\}$), and it achieved 94.15% accuracy in feasibility classification. The remarkable accuracy indicates that SatGNN is highly effective at learning the constraints of the ONTS problem. This performance is expected as the feasibility classification task requires the model to learn linear classification boundaries, which are derived from the problem’s linear constraints. In other words, the model learns the indicator function of a convex set (considering the problem’s LP relaxation), for which neural networks are known to be suitable.

The second set of experiments (Sec. 6.3) aimed to predict optimal solutions to ONTS instances using SatGNN. More specifically, SatGNN was trained to predict the biases of the binary variables for multiple instances of the ONTS problem. Two training approaches were explored: one trained with a single target (the quasi-optimal solution) and another trained with multiple solutions (the best feasible solutions found). Both approaches showed promising results, with no signs of overfitting, as indicated by the small difference between validation and test set performance.

Both models achieved similar accuracy results on a per-variable basis, with the model trained with multiple solutions having a slight advantage of 2 p.p. on average. Nevertheless, as illustrated by Figure 9, both models show similar distributions, always correctly predicting more than 75% of the variables in all instances. Interestingly, the model trained with multiple solutions tended to be more confident in its predictions, particularly for the binary variables. This increased confidence could be valuable in guiding the optimization process, as it can be understood as an indication of the model’s certainty about its predictions.

Although the results from the second set of experiments suggest that SatGNN can effectively predict optimal or near-optimal solutions to ONTS instances, we put the models to the test through a third set of experiments (Sec. 6.4) in which we build (mat)heuristic solutions based on the output of the SatGNN trained for optimality prediction. Namely, as detailed in Sec. 5.4, three approaches were evaluated for applying the model’s output:

warm-start, early fix, and trust region. Each approach is optimized for finding the best solution, giving the time budget (2 min.) and reducing the time to find a feasible solution (2 separate experiments). First, we notice that in all direct comparisons, the SatGNN model trained with multiple solutions resulted in better heuristic solutions than the model trained with a single solution, as the results of the second set of experiments suggested.

Early-fixing the partial solution provided by the SatGNN models yielded the best results overall, showing a 43% gain in the evaluation of the expected objective value within the time budget, and a 35% reduction in the evaluation of the expected time to find a feasible solution. The trust region approach, which can be seen as a relaxation of early-fixing, was comparable to early-fixing in terms of the best solution found within the time limit. However, overall, it seems to provide a worse trade-off, as both early-fixing and trust region do not provide optimality guarantees, but trust region takes longer to find good solutions to the ONTS problem. Using the SatGNN models to warm-start the SCIP solver significantly reduced the time needed to find a feasible solution (in comparison to the baseline), but it was worse overall than the two other approaches.

Across all experiments, a highlight of the GNNs’ performance was the generalization to larger instances. This characteristic is essential for problems with varying instance size and is not trivially achievable with traditional deep-learning models. However, it is also key for tasks with higher acquisition costs for certain problem sizes, as in the case of the ONTS problem. Large instances of optimization problems are the ones that drive the research of heuristic solutions, as they are usually more expensive to solve; however, as we need to solve instances to create a training dataset, it often becomes infeasible to train the deep learning models on large instances. In this context, the generalization results from our experiments suggest that GNNs can be effectively trained using easy (cheap to acquire) instances of the problem, and be used on the hard instances of interest.

Generating reliable instances to train and evaluate solvers is a key point in such a project, as the confidence of the results depends on how carefully instances are designed to compose the test set (Smith-Miles and Bowly, 2015). Data generation is particularly critical when historical data is not available (Bengio et al., 2021), which motivates the research on algorithms for generating reliable instances (Smith-Miles and Bowly, 2015; Malitsky et al., 2016). We follow the problem definition of the FloripaSat-I mission (Marcelino et al., 2020) to define the parameters’ ranges used to sample new instances uniformly. Although the data generation process is reliable, it still depends on the optimization of every randomly generated instance. Furthermore, as we restrict ourselves to feasible instances, we only add to the dataset instances for which the solver could find at least one feasible solution within the time budget, effectively restricting our data to an easier sub-problem (Yehuda et al., 2020).

Finally, we underscore that the computational gains observed in terms of time to find a feasible solution (see, for example, Figure 12b) are expected to directly translate to the deployed application. First, because dataset generation and training, which are indeed costly, as discussed above, are performed offline. Then, generating a partial solution with SatGNN takes a negligible amount of time compared to solving the instances. Beyond that, the inference time scales linearly with the size of the graph, which is directly proportional to the

instance size (number of variables and constraints). Given that the problem is combinatorial, we can expect the SatGNN inference cost to be totally dominated by the cost of solving the instances for realistic instances.

8. Conclusion and Future Research

This work has proposed a novel approach to tackle the ONTS problem using graph neural networks. More specifically, we investigated whether a GNN can learn the structure of the problem and be used to build effective heuristic solutions. Our experiments showed that our proposed architecture, SatGNN, successfully classifies the feasibility of candidate solutions and is also able to provide reliable partial solutions to varied instances of the ONTS problem. Not only was the model able to generalize to unseen instances, but it also showed promising results on out-of-distribution instances, which were larger than the ones seen during training. This outcome shows how the inherent symmetries of graph neural networks make them suitable for dealing with the structures of the optimization problem.

By leveraging partial solutions provided by the SatGNN model, we built heuristic solutions to the ONTS problem that outperformed the SCIP solver in hard instances. More specifically, by fixing the variable assignment from the partial solution provided by SatGNN, our heuristics were able to improve up to 43% in the expected objective value of the best solution found within the time budget and reduce up to 35% in the expected time to find a feasible solution.

Overall, the experiments presented in this paper showcase the potential of GNNs for addressing the ONTS problem. The high accuracy in feasibility classification, the ability to predict optimal solutions, and the effectiveness of SatGNN-based heuristics suggest that our learning-based approach yields a strong primal heuristic that has the potential to significantly improve the performance of exact methods for complex space mission scheduling problems.

8.1. Future Work

Given the nature of the ONTS problem, generating large, difficult instances is a significant challenge. Although realistic, the instance generation method used in this work is limited regarding instance difficulty by the time budget. One possible future research direction is to leverage easier instances, combining job parameters to generate large feasible instances that extrapolate the time budget restriction. This line also leads to a non-trivial application of machine learning models for combinatorial optimization: guiding the search for difficult instances. More specifically, using the SatGNN for feasibility classification could significantly alleviate the cost of generating larger feasible instances.

Reinforcement learning (RL) has shown promise for learning-based approaches to combinatorial optimization (Jalali Khalil Abadi et al., 2024; O’Malley et al., 2023; Waubert de Puiseau et al., 2022). It has also been explored in settings close to ONTS, such as Earth observation satellite scheduling (He et al., 2022; Jacquet et al., 2024). The possibility of taking sequential decisions makes it easier to incorporate feasibility information into the model. In our case, it could enable the fixing of larger partial solutions without rendering the resulting mathematical program infeasible, reducing the computational cost. It may also

reduce dataset-generation effort by learning from interaction rather than labeled optimal solutions. However, developing an RL-based primal heuristic requires careful reward design and an interactive environment, which is substantially different from our supervised learning approach; we therefore leave a full RL solution to future work.

Future works naturally entail the integration of the proposed methodology into exact methods tailored to the ONTS problem. State-of-the-art branch-and-price algorithms (Rigo et al., 2022) could significantly benefit from better initial solutions. The explainability aspects could be further developed to provide deeper insights into solution quality and guide the generation of columns for specific jobs. Additionally, the approach could be adapted to handle dynamic scheduling scenarios where task requirements evolve over time, requiring online learning to update the parameters of the SatGNN. Also, the principles developed here could be extended to related space mission optimization problems, including constellation management and multi-satellite resource allocation challenges.

Acknowledgments

The authors acknowledge support from CNPq (*Conselho Nacional de Desenvolvimento Científico e Tecnológico*) under grant numbers 150281/2022-6 and 404576/2021-4 as well as FAPESC under grant number 2021TR001851.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author(s) used generative AI tools in order to perform light editing of the text. After using these tools/services, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the publication.

Appendix A. Symbols in the ONTS mathematical programming model

The table below compiles the sets, variables, and parameters of the ONTS MILP formulation.

Symbol	Description
<i>Sets</i>	
$\mathcal{J}$	Tasks assigned to the nanosatellite $\{1, \dots, J\}$
$\mathcal{T}$	Discrete time periods within the planning horizon $\{1, \dots, T\}$
<i>Decision Variables</i>	
$x_{j,t}$	Binary: 1 if task j is executed at time t , 0 otherwise
$\phi_{j,t}$	Binary: 1 if task j starts at time t , 0 otherwise
<i>Parameters</i>	
$u_{j,t}$	Utility or mission value derived from executing task j at time t
$w_j^{\min}$	Minimum start time window for task j
$w_j^{\max}$	Maximum start time window for task j
$p_j^{\min}$	Minimum time between consecutive starts of task j
$p_j^{\max}$	Maximum time between consecutive starts of task j
$t_j^{\min}$	Minimum execution time of task j
$t_j^{\max}$	Maximum execution time of task j
$y_j^{\min}$	Minimum number of start-ups for task j
$y_j^{\max}$	Maximum number of start-ups for task j
b_t	Energy balance at time t
i_t	Total current at time t
SoC_t	State-of-charge of the battery at time t
r_t	Power input at time t (from solar panels)
q_j	Power consumption of task j per time slot
V_b	Nominal battery voltage
e	Battery efficiency during charging and discharging
Q	Nominal capacity of the battery
ρ	Minimum energy level allowed for the battery
γ	Maximum allowed current rate for charging or discharging the battery

Appendix B. Random instance generation

For any particular mission size and orbital length, the main objective is to generate a realistic ONTS case using random data. We have taken the FloripaSat-I mission as a reference for instance generation, which has an altitude of 628 kilometers and an orbital period of 97.2 minutes Marcelino et al. (2020). The battery-related parameters are fixed for all instances as

$$e \leftarrow 0.9$$

$$Q \leftarrow 5$$

$$\gamma \leftarrow 5$$

$$V_b \leftarrow 3.6$$

$$\rho \leftarrow 0.0$$

The attitude considered here is the Nadir, in which the satellite turns at the same rate around the Earth, so one side (or axis) always faces the Earth’s surface. This analytical model then utilizes a rotation matrix to simulate the satellite’s dynamics and can be adapted for larger or different geometries by adjusting the normal vectors representing the body.

To generate realistic power input vectors $\mathbf{r}$, we use 2 years of historical data of solar irradiance in orbit. More precisely, a window is randomly drawn from the historical data, and an analytical model is used to determine the power input vector. Once orbits are stable and solar flux constant – $1360W/m^2$ – one can calculate this vector by knowing the spacecraft orbit, attitude – its kinematics – and size Filho et al. (2020).

The remaining parameters are drawn uniformly, given handcrafted limits to increase the feasibility rate. Specifically, given the number of tasks J and the number of time units T , the parameters are drawn as

$$\begin{aligned}
u_j &\leftarrow \mathcal{U}(1, J) \\
q_j &\leftarrow \mathcal{U}(0.3, 2.5) \\
y_j^{\min} &\leftarrow \mathcal{U}(1, \lceil T/45 \rceil) \\
y_j^{\max} &\leftarrow \mathcal{U}(y_j^{\min}, \lceil T/15 \rceil) \\
t_j^{\min} &\leftarrow \mathcal{U}(1, \lceil T/10 \rceil) \\
t_j^{\max} &\leftarrow \mathcal{U}(t_j^{\min}, \lceil T/4 \rceil) \\
p_j^{\min} &\leftarrow \mathcal{U}(t_j^{\min}, \lceil T/4 \rceil) \\
p_j^{\max} &\leftarrow \mathcal{U}(p_j^{\min}, T) \\
w_j^{\min} &\leftarrow \mathcal{U}(0, \lceil T/5 \rceil) \\
w_j^{\max} &\leftarrow \mathcal{U}(\lfloor T - \lceil T/5 \rceil \rfloor, T).
\end{aligned}$$

Appendix C. Symbols for SatGNN

The table below provides a summary of the main symbols used in the definition of SatGNN and the related methods.

Symbol	Description
<i>Graph representation</i>	
V_{var}	Nodes associated to variables; $ V_{\text{var}} = n$.
V_{con}	Nodes associated to constraints; $ V_{\text{con}} = m$.
E	Edges between variable nodes and constraint nodes.
$w(e)$	Weight of edge $e \in E$.
$\mathcal{N}(v)$	Neighborhood of node $v \in V$.
<i>Features and hidden representations</i>	
d_{in}	Number of input features for each node.
$\mathbf{f}^v$	Input feature vector of node v .
F_{var}	Matrix of hidden features for variable nodes.
F_{con}	Matrix of input features for constraint nodes.
$L \in \mathbb{N}$	Number of graph convolution layers.
$d \in \mathbb{N}$	Number of hidden features for each node.
$\mathbf{h}_v^{(l)}$	Hidden feature vector of node v at layer $l \in \{0, \dots, L\}$.
$H_{\text{var}}^{(l)}$	Matrix of hidden features for variable nodes at layer l .
$H_{\text{con}}^{(l)}$	Matrix of hidden features for constraint nodes at layer l .
d_{out}	Number of output features for each node.
<i>Fully-connected modules</i>	
NN_{var}	Encoder for variable-node features (produces $\mathbf{h}^{(0)}$ on V_{var}).
NN_{con}	Encoder for constraint-node features (produces $\mathbf{h}^{(0)}$ on V_{con}).
NN_{out}	Output head applied to $\mathbf{h}^{(L)}$ (task-dependent output dimension).
<i>Training</i>	
$\mathcal{L}$	Loss function.
$\mathcal{D}$	Dataset of instances and candidate solutions.
$p(\hat{\mathbf{z}} \in Z I)$	True “probability” that the assignment $\hat{\mathbf{z}}$ is feasible for instance I . Because there is no stochasticity, it can either be 0 or 1.
$\hat{p}(\hat{\mathbf{z}} \in Z I)$	Estimated “probability” (predicted variable bias) that the assignment $\hat{\mathbf{z}}$ is feasible for instance I .
$\hat{p}(\mathbf{z}^* = 1 I)$	Predicted variable biases (towards 1) of an optimal solution for instance I .
<i>Heuristics</i>	
$\kappa(z I)$	Confidence the model has in each variable bias prediction. Defined as the maximum over the predicted marginal probabilities.
$\mathcal{C}_I^N$	set of indices of the $N \in \mathbb{N}$ variables that the model is most confident with with respect to its prediction for instance I .
Δ	Size of the trust region around a predicted partial assignment.

Appendix D. Hyperparameter Tuning

We conduct a random search over the hyperparameters listed in Table 5 and evaluate each configuration on the validation set. For each configuration we train SatGNN using the best

solution (BS) and the multiple solution (MS) approaches on the training set. The performance is measured as the best validation loss observed in 10 epochs. The best configurations seen for each training method is reported in Table 5.

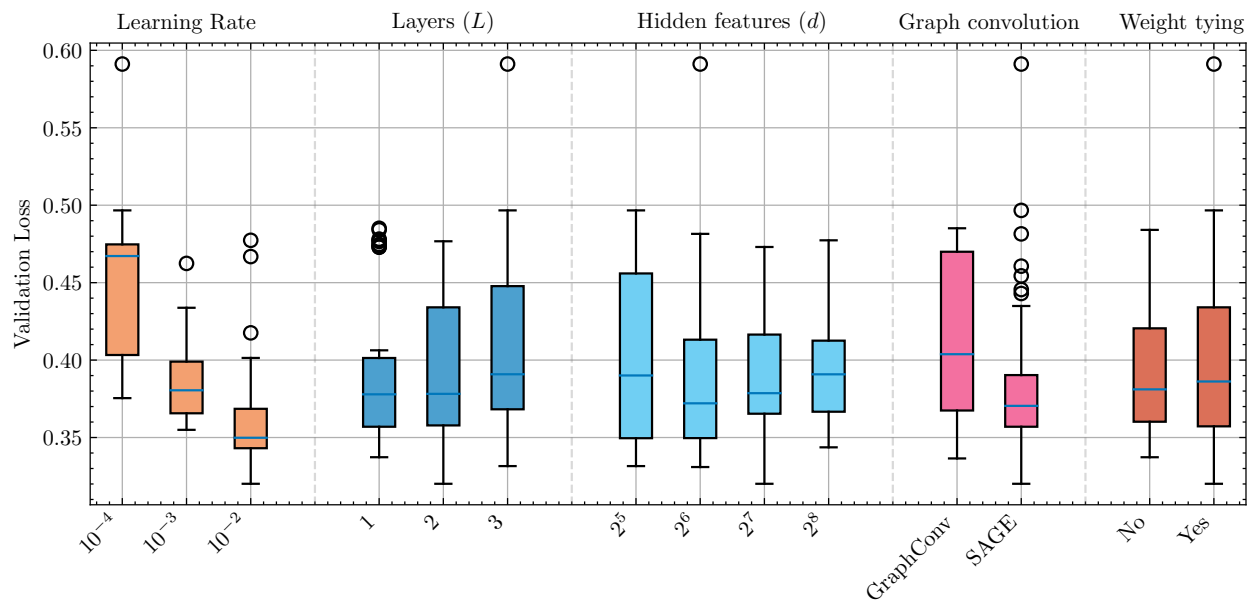
We evaluate the relevance of tuning each hyperparameter by measuring how sensitive the validation loss of SatGNN is to the different values in search space. The distribution of the validation loss given each hyperparameter choice is illustrated in Figure D.15. For each hyperparameter, we test whether validation losses differ across its range using the nonparametric Kruskal–Wallis H test (MacFarland and Yates, 2016). We apply the test independently for the BS and the MS trainings. The only hyperparameters that had a statistically significant ($p \ll 0.05$) impact in the validation loss were the learning rate and the graph convolutional operator, for both training approaches. All other hyperparameters did not achieve significance under this criterion, although the size of the hidden feature vectors d had a strong impact ($H = 6.84$ and $p < 0.08$) when training with MS.

We refer the reader to our code repository⁷ for further details, as we have made all our hyperparameter tuning implementation and results available.

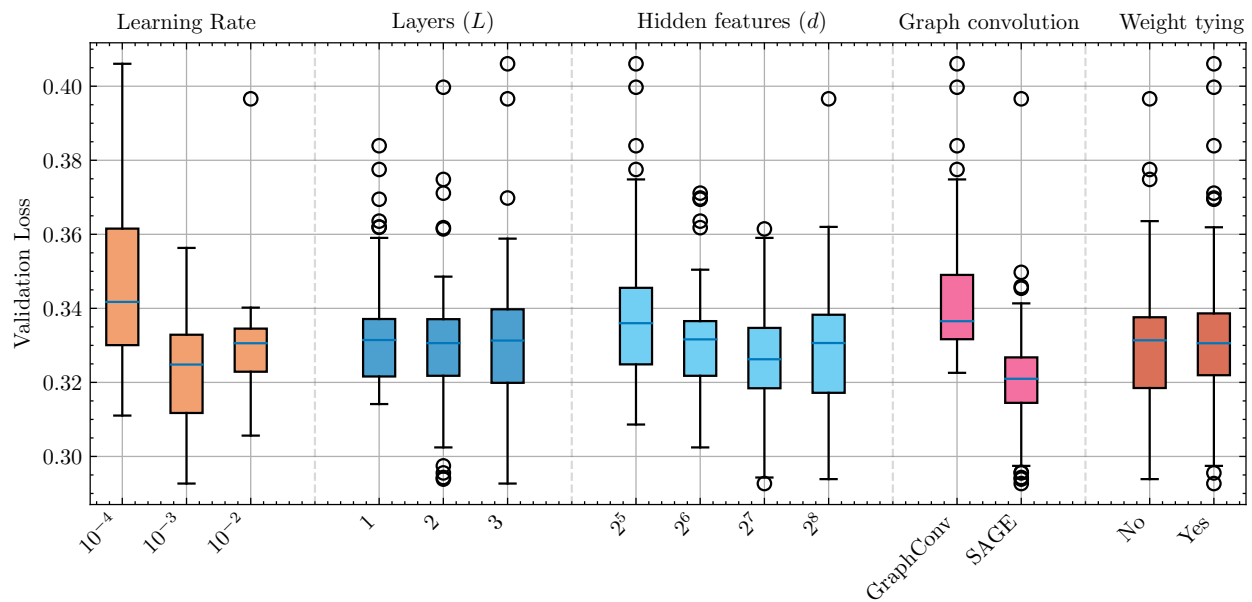
References

- Bengio, Y., Lodi, A., Prouvost, A., 2021. Machine learning for combinatorial optimization: A methodological tour d’horizon. *European Journal of Operational Research* 290, 405–421. doi:10.1016/j.ejor.2020.07.063.
- Bestuzheva, K., Besançon, M., Chen, W.K., Chmiela, A., Donkiewicz, T., Doornmalen, J.v., Eifler, L., Gaul, O., Gamrath, G., Gleixner, A., Gottwald, L., Graczyk, C., Halbig, K., Hoen, A., Hojny, C., Hulst, R.v.d., Koch, T., Lübbecke, M., Maher, S.J., Matter, F., Mühmer, E., Müller, B., Pfetsch, M.E., Rehfeldt, D., Schlein, S., Schlösser, F., Serrano, F., Shinano, Y., Sofranac, B., Turner, M., Vigerske, S., Wegscheider, F., Wellner, P., Weninger, D., Witzig, J., 2021. The SCIP Optimization Suite 8.0. ZIB-Report 21-41. Zuse Institute Berlin. URL: <http://nbn-resolving.de/urn:nbn:de:0297-zib-85309>.
- Boschetti, M.A., Maniezzo, V., 2022. Matheuristics: using mathematics for heuristic design. *4OR* 20, 173–208. doi:10.1007/s10288-022-00510-8.
- Camponogara, E., Seman, L.O., Rigo, C.A., Filho, E.M., Ribeiro, B.F., Bezerra, E.A., 2022. A continuous-time formulation for optimal task scheduling and quality-of-service assurance in nanosatellites. *Computers & Operations Research* 147, 105945. doi:10.1016/j.cor.2022.105945.
- Cappart, Q., Chételat, D., Khalil, E., Lodi, A., Morris, C., Veličković, P., 2022. Combinatorial optimization and reasoning with graph neural networks. doi:10.48550/arXiv.2102.09544.

⁷<https://github.com/brunompacheco/sat-gnn>



(a) Best Solution



(b) Multiple Solutions

Figure D.15: Validation loss by hyperparameter choice in our hyperparameter tuning experiments.

- Cui, K., Song, J., Zhang, L., Tao, Y., Liu, W., Shi, D., 2023. Event-triggered deep reinforcement learning for dynamic task scheduling in multisatellite resource allocation. *IEEE Transactions on Aerospace and Electronic Systems* 59, 3766–3777. doi:10.1109/TAES.2022.3231239.
- Dalin, L., Haijiao, W., Zhen, Y., Yanfeng, G., Shi, S., 2021. An online distributed satellite cooperative observation scheduling algorithm based on multiagent deep reinforcement learning. *IEEE Geoscience and Remote Sensing Letters* 18, 1901–1905. doi:10.1109/LGRS.2020.3009823.
- Ding, J.Y., Zhang, C., Shen, L., Li, S., Wang, B., Xu, Y., Song, L., 2020. Accelerating primal solution findings for mixed integer programs based on solution prediction, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 1452–1459. Issue: 02.
- Feng, X., Li, Y., Xu, M., 2024. Multi-satellite cooperative scheduling method for large-scale tasks based on hybrid graph neural network and metaheuristic algorithm. *Advanced Engineering Informatics* 60, 102362. doi:10.1016/j.aei.2024.102362.
- Filho, E.M., Nicolau, V.d.P., Paiva, K.V.d., Possamai, T.S., 2020. A comprehensive attitude formulation with spin for numerical model of irradiance for CubeSats and Picosats. *Applied Thermal Engineering* 168, 114859. doi:10.1016/j.applthermaleng.2019.114859.
- Gasse, M., Chételat, D., Ferroni, N., Charlin, L., Lodi, A., 2019. Exact combinatorial optimization with graph convolutional neural networks. *Advances in Neural Information Processing Systems* 32.
- Guo, W., Vanhoucke, M., Coelho, J., 2023. A prediction model for ranking branch-and-bound procedures for the resource-constrained project scheduling problem. *European Journal of Operational Research* 306, 579–595. doi:10.1016/j.ejor.2022.08.042.
- Hamilton, W., Ying, Z., Leskovec, J., 2017. Inductive representation learning on large graphs. doi:10.48550/arXiv.1706.02216. arXiv:1706.02216.
- Han, Q., Yang, L., Chen, Q., Zhou, X., Zhang, D., Wang, A., Sun, R., Luo, X., 2023. A GNN-guided predict-and-search framework for mixed-integer linear programming. doi:10.48550/arXiv.2302.05636. arXiv:2302.05636.
- He, Y., Xing, L., Chen, Y., Pedrycz, W., Wang, L., Wu, G., 2022. A generic Markov decision process model and reinforcement learning method for scheduling agile Earth observation satellites. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 52, 1463–1474. doi:10.1109/TSMC.2020.3020732.
- Hopfield, J.J., Tank, D.W., 1985. ‘Neural’ computation of decisions in optimization problems. *Biological Cybernetics* 52, 141–152. doi:10.1007/BF00339943.

- Huang, Y., Mu, Z., Wu, S., Cui, B., Duan, Y., 2021. Revising the observation satellite scheduling problem based on deep reinforcement learning. *Remote Sensing* 13. doi:10.3390/rs13122377.
- Jacquet, A., Infantes, G., Meuleau, N., Benazera, E., Roussel, S., Baudoui, V., Guerra, J., 2024. Earth observation satellite scheduling with graph neural networks. doi:10.48550/arXiv.2408.15041.
- Jalali Khalil Abadi, Z., Mansouri, N., Javidi, M.M., 2024. Deep reinforcement learning-based scheduling in distributed systems: A critical review. *Knowledge and Information Systems* 66, 5709–5782. doi:10.1007/s10115-024-02167-7.
- Karimi-Mamaghan, M., Mohammadi, M., Meyer, P., Karimi-Mamaghan, A.M., Talbi, E.G., 2022. Machine learning at the service of meta-heuristics for solving combinatorial optimization problems: A state-of-the-art. *European Journal of Operational Research* 296, 393–422. doi:10.1016/j.ejor.2021.04.032.
- Kenworthy, L., Nayak, S., Chin, C., Balakrishnan, H., 2022. NICE: Robust scheduling through reinforcement learning-guided integer programming. *Proceedings of the AAAI Conference on Artificial Intelligence* 36, 9821–9829. doi:10.1609/aaai.v36i9.21218.
- Khalil, E., Dai, H., Zhang, Y., Dilkina, B., Song, L., 2017. Learning combinatorial optimization algorithms over graphs, in: *Advances in Neural Information Processing Systems*. URL: https://proceedings.neurips.cc/paper_files/paper/2017/hash/d9896106ca98d3d05b8cbdf4fd8b13a1-Abstract.html.
- Khalil, E.B., Morris, C., Lodi, A., 2022. MIP-GNN: A data-driven framework for guiding combinatorial solvers. *Proceedings of the AAAI Conference on Artificial Intelligence* 36, 10219–10227. doi:10.1609/aaai.v36i9.21262. number: 9.
- Kingma, D.P., Ba, J., 2015. Adam: A method for stochastic optimization, in: Bengio, Y., LeCun, Y. (Eds.), *3rd International Conference on Learning Representations (ICLR)*, San Diego, CA. doi:10.48550/arXiv.1412.6980.
- Kipf, T.N., Welling, M., 2017. Semi-supervised classification with graph convolutional networks. doi:10.48550/arXiv.1609.02907. arXiv:1609.02907.
- Kruber, M., Lübbecke, M.E., Parmentier, A., 2017. Learning when to use a decomposition, in: Salvagnin, D., Lombardi, M. (Eds.), *Integration of AI and OR Techniques in Constraint Programming*, Cham. pp. 202–210. doi:10.1007/978-3-319-59776-8_16.
- Li, J., Wu, G., Liao, T., Fan, M., Mao, X., Pedrycz, W., 2023. Task scheduling under a novel framework for data relay satellite network via deep reinforcement learning. *IEEE Transactions on Vehicular Technology* 72, 6654–6668. doi:10.1109/TVT.2022.3233358.

- Lucia, B., Denby, B., Manchester, Z., Desai, H., Ruppel, E., Colin, A., 2021. Computational nanosatellite constellations: Opportunities and challenges. *GetMobile: Mobile Comp. and Comm.* 25, 16–23. doi:10.1145/3471440.3471446.
- MacFarland, T.W., Yates, J.M., 2016. Kruskal–Wallis H-Test for oneway analysis of variance (ANOVA) by ranks, in: MacFarland, T.W., Yates, J.M. (Eds.), *Introduction to Nonparametric Statistics for the Biological Sciences Using R*. Springer International Publishing, Cham, pp. 177–211. doi:10.1007/978-3-319-30634-6_6.
- de Magalhães Taveira-Gomes, T.S., 2017. Reinforcement Learning for Primary Care and Appointment Scheduling. Ph.D. thesis. Faculdade de Engenharia da Universidade do Porto.
- Malitsky, Y., Merschformann, M., O’Sullivan, B., Tierney, K., 2016. Structure-preserving instance generation, in: Festa, P., Sellmann, M., Vanschoren, J. (Eds.), *Learning and Intelligent Optimization*, Cham. pp. 123–140. doi:10.1007/978-3-319-50349-3_9.
- Marcelino, G.M., Vega-Martinez, S., Seman, L.O., Kessler Slongo, L., Bezerra, E.A., 2020. A critical embedded system challenge: The FloripaSat-1 mission. *IEEE Latin America Transactions* 18, 249–256. doi:10.1109/TLA.2020.9085277.
- Nagel, G.W., de Moraes Novo, E.M.L., Kampel, M., 2020. Nanosatellites applied to optical Earth observation: a review. *Ambiente e Agua – An Interdisciplinary Journal of Applied Science* 15, 1. doi:10.4136/ambi-agua.2513.
- Nair, V., Alizadeh, M., others, 2020. Neural large neighborhood search, in: *Learning Meets Combinatorial Algorithms at NeurIPS2020*.
- Niu, X., Tang, H., Wu, L., 2018. Satellite scheduling of large areal tasks for rapid response to natural disaster using a multi-objective genetic algorithm. *International Journal of Disaster Risk Reduction* 28, 813–825. doi:10.1016/j.ijdr.2018.02.013.
- O’Malley, C., de Mars, P., Badesa, L., Strbac, G., 2023. Reinforcement learning and mixed-integer programming for power plant scheduling in low carbon systems: Comparison and hybridisation. *Applied Energy* 349, 121659. doi:10.1016/j.apenergy.2023.121659.
- Pacheco, B.M., Seman, L.O., Camponogara, E., 2023a. Deep-learning-based early fixing for gas-lifted oil production optimization: Supervised and weakly-supervised approaches. *arXiv:2309.00197*.
- Pacheco, B.M., Seman, L.O., Rigo, C.A., Camponogara, E., 2023b. [dataset] FloripaSat MILPs. doi:10.5281/zenodo.8356798.
- Parmentier, A., T’Kindt, V., 2023. Structured learning based heuristics to solve the single machine scheduling problem with release times and sum of completion times. *European Journal of Operational Research* 305, 1032–1041. doi:10.1016/j.ejor.2022.06.040.

- Rigo, C.A., Morsch Filho, E., Seman, L.O., Loures, L., Leithardt, V.R.Q., 2023. Instance and data generation for the offline nanosatellite task scheduling problem. *Data* 8. doi:10.3390/data8030062.
- Rigo, C.A., Seman, L.O., Camponogara, E., Filho, E.M., Bezerra, E.A., 2021a. A nanosatellite task scheduling framework to improve mission value using fuzzy constraints. *Expert Systems with Applications* 175, 114784. doi:10.1016/j.eswa.2021.114784.
- Rigo, C.A., Seman, L.O., Camponogara, E., Filho, E.M., Bezerra, E.A., 2021b. Task scheduling for optimal power management and quality-of-service assurance in CubeSats. *Acta Astronautica* 179, 550–560. doi:10.1016/j.actaastro.2020.11.016.
- Rigo, C.A., Seman, L.O., Camponogara, E., Filho, E.M., Bezerra, E.A., Munari, P., 2022. A branch-and-price algorithm for nanosatellite task scheduling to improve mission quality-of-service. *European Journal of Operational Research* 303, 168–183. doi:10.1016/j.ejor.2022.02.040.
- Rouzot, J., Gobert, J., Artigues, C., Boyer, R., Camps, F., Garnier, P., Hebrard, E., Lopez, P., 2024. Scheduling onboard tasks of the NIMPH nanosatellite, in: Library, S.D. (Ed.), 13th International Conference on Operations Research and Enterprise Systems (ICORES), Rome, Italy. URL: <https://laas.hal.science/hal-04430171>.
- Saeed, N., Elzanaty, A., Almorad, H., Dahrouj, H., Al-Naffouri, T.Y., Alouini, M.S., 2020. Cubesat communications: Recent advances and future challenges. *IEEE Communications Surveys & Tutorials* 22, 1839–1862. doi:10.1109/COMST.2020.2990499.
- Seman, L.O., Ribeiro, B.F., Rigo, C.A., Filho, E.M., Camponogara, E., Leonardi, R., Bezerra, E.A., 2022. An energy-aware task scheduling for quality-of-service assurance in constellations of nanosatellites. *Sensors* 22. doi:10.3390/s22103715.
- Seman, L.O., Rigo, C.A., Camponogara, E., Bezerra, E.A., dos Santos Coelho, L., 2023. Explainable column-generation-based genetic algorithm for knapsack-like energy aware nanosatellite task scheduling. *Applied Soft Computing* 144, 110475. doi:10.1016/j.asoc.2023.110475.
- Shiroma, W.A., Martin, L.K., Akagi, J.M., Akagi, J.T., Wolfe, B.L., Fewell, B.A., Ohta, A.T., 2011. CubeSats: A bright future for nanosatellites. *Central European Journal of Engineering* 1, 9–15. doi:10.2478/s13531-011-0007-8.
- Smith, K.A., 1999. Neural networks for combinatorial optimization: A review of more than a decade of research. *INFORMS Journal on Computing* 11, 15–34. doi:10.1287/ijoc.11.1.15.
- Smith-Miles, K., Bowly, S., 2015. Generating new test instances by evolving in instance space. *Computers & Operations Research* 63, 102–113. doi:10.1016/j.cor.2015.04.022.

- Tassel, P., Gebser, M., Schekotihin, K., 2021. A reinforcement learning environment for job-shop scheduling. *arXiv:2104.03760*.
- Wang, J., Demeulemeester, E., Qiu, D., 2016. A pure proactive scheduling algorithm for multiple earth observation satellites under uncertainties of clouds. *Computers & Operations Research* 74, 1–13. doi:10.1016/j.cor.2016.04.014.
- Wang, J., Zhu, X., Qiu, D., Yang, L.T., 2013. Dynamic scheduling for emergency tasks on distributed imaging satellites with task merging. *IEEE Transactions on Parallel and Distributed Systems* 25, 2275–2285. doi:10.1109/TPDS.2013.156.
- Wang, L., Pan, Z., Wang, J., 2021. A review of reinforcement learning based intelligent optimization for manufacturing scheduling. *Complex System Modeling and Simulation* 1, 257–270. doi:10.23919/CSMS.2021.0027.
- Waubert de Puiseau, C., Meyes, R., Meisen, T., 2022. On reliability of reinforcement learning based production scheduling systems: A comparative survey. *Journal of Intelligent Manufacturing* 33, 911–927. doi:10.1007/s10845-022-01915-2.
- Wilcoxon, F., 1945. Individual comparisons by ranking methods. *Biometrics Bulletin* 1, 80–83. URL: <http://www.jstor.org/stable/3001968>.
- Yang, Y., Boland, N., Dilkina, B., Savelsbergh, M., 2022. Learning generalized strong branching for set covering, set packing, and 0–1 knapsack problems. *European Journal of Operational Research* 301, 828–840. doi:10.1016/j.ejor.2021.11.050.
- Yehuda, G., Gabel, M., Schuster, A., 2020. It’s not what machines can learn, it’s what we cannot teach, in: *Proceedings of the 37th International Conference on Machine Learning*, PMLR. pp. 10831–10841. URL: <https://proceedings.mlr.press/v119/yehuda20a.html>. iSSN: 2640-3498.
- Zhang, C., Song, W., Cao, Z., Zhang, J., Tan, P.S., Chi, X., 2020. Learning to dispatch for job shop scheduling via deep reinforcement learning, in: *Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., Lin, H. (Eds.), Advances in Neural Information Processing Systems*, Curran Associates, Inc.. pp. 1621–1632. URL: https://proceedings.neurips.cc/paper_files/paper/2020/file/11958dfce29b6709f48a9ba0387a2431-Paper.pdf.
- Zhang, J., Liu, C., Li, X., Zhen, H.L., Yuan, M., Li, Y., Yan, J., 2023. A survey for solving mixed integer programming via machine learning. *Neurocomputing* 519, 205–217. doi:10.1016/j.neucom.2022.11.024.